

Minimum Spanning Tree





Review – lecture 7

Greedy Algorithms

Greedy idea: Perform an operation that has a local justification (short-range view).

General structure:

1. Preprocessing (usually sorting)
2. Build the solution gradually, step after step.

Review – lecture 7

General proof structure:

1. **Prove the Greedy choice property:**

If the algorithm does X then there exists an optimal solution that does X.

Proof structure: Given a solution that does not do X:

- Either show it is not optimal
- Or show it can be modified to do X without hurting its performance.

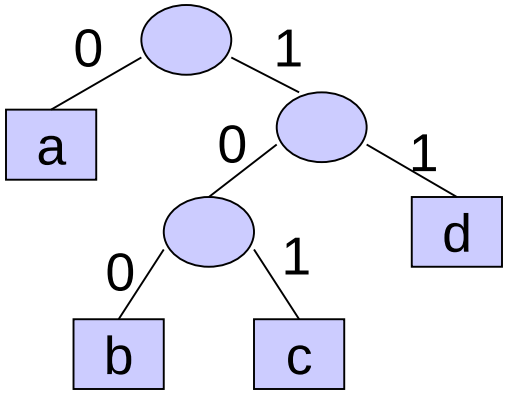
Review – lecture 7

2. Use induction to extend the greedy choice property for any prefix of the solution:

Claim: For any $k \geq 0$, after k steps, there exists an optimal solution that agrees with the first k steps of Greedy.
Base case: k=0 – empty solution.
Step: Consider the situation after k-1 steps. There exists an optimal solution that agrees with the partial solution of Greedy at this stage (this is the induction hypothesis). Now argue something about the remaining problem and how it is connected to the prefix that was already processed. Use the GCP for the induction step.

This part depends on the problem…

Review – lecture 7



Huffman Code.

Input: text over an alphabet C

Output: prefix-free binary code.

Goal: minimal total length of the encoded text.

- Prefix-free codes have unique linear time encoding and decoding.
- A prefix-free code is represented by a binary tree, in which the leaves correspond to characters of C.

The cost of a tree = $\sum_{a \in C} p(a) \cdot d_T(a)$.

Example: Assume p(a)=0.4, p(b)=0.1, p(c)=0.2, p(d)=0.3

cost(T)=0.4·1+0.1·3+0.2·3+0.3·2= 1.9

Review – lecture 7

Huffman Code.

Algorithm: Build the tree from bottom to top

Start with $|C|$ leaves labeled by the characters' frequencies $p(a)$.

In each iteration, add one node as the parent of the two roots with the lowest labels, u_1, u_2.

Determine $p(v)=p(u_1)+p(u_2)$.

Proof idea:

1. Every optimal prefix-free code is represented by a full tree.
2. GCP: There exists an optimal prefix-free code, in which the lowest frequencies are siblings in maximal depth.
3. Induction on $|C|$.

Minimum Spanning Tree

An optimization problem on graphs, that can be solved optimally by a greedy algorithm.



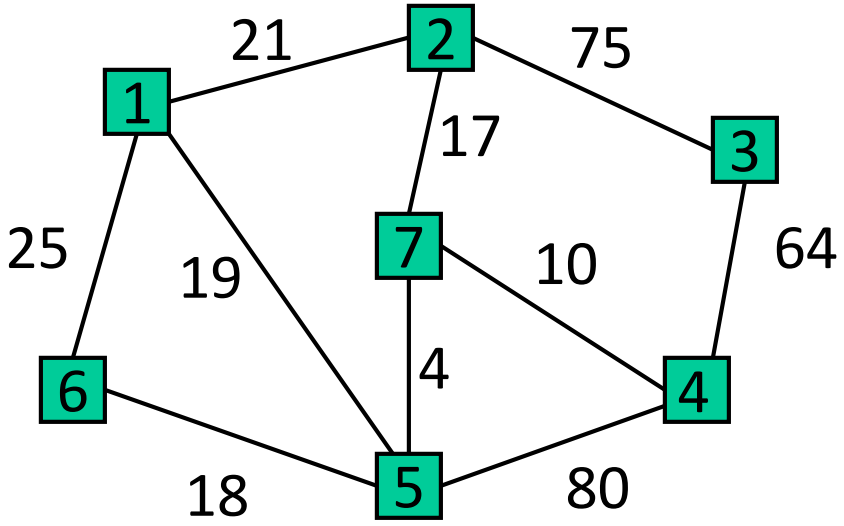


Minimum Spanning Tree (MST)

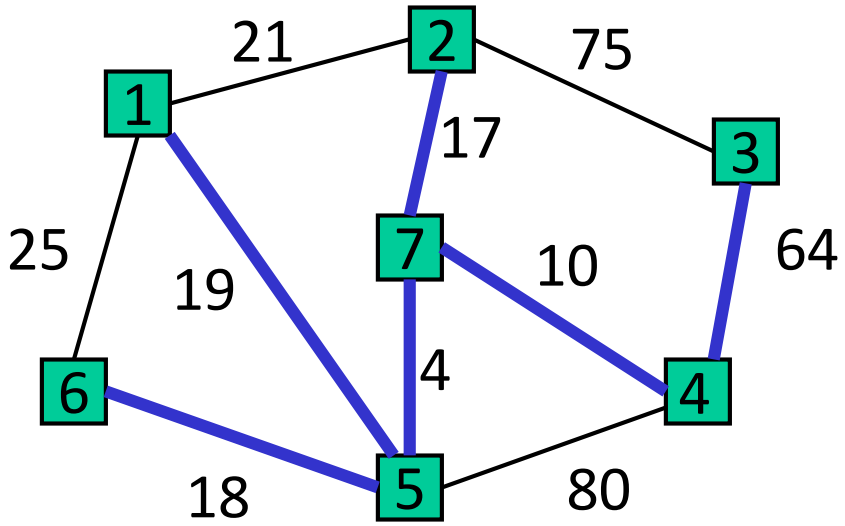
- Input: An undirected connected graph.

  Each edge has a non-negative cost.

- Output: A minimal-cost subset of edges that will keep the graph connected (must be a ST).



Example of a Minimum Spanning Tree



Price of this tree = 18+19+4+10+17+64.

It is cheapest among all spanning trees of this graph.

Minimum Spanning Tree Problem

- Input: Undirected connected graph G = (V,E) and a cost function c: E→$R_{\geq 0}$. c(e) is the cost of edge e.
- Output: A spanning tree T with minimum total cost. That is, T that minimizes

$$c(T) = \sum_{e \in T} c(e)$$

- Another formulation: Remove from G edges with maximal total cost, such that we are left with a tree.

Minimum Spanning Tree

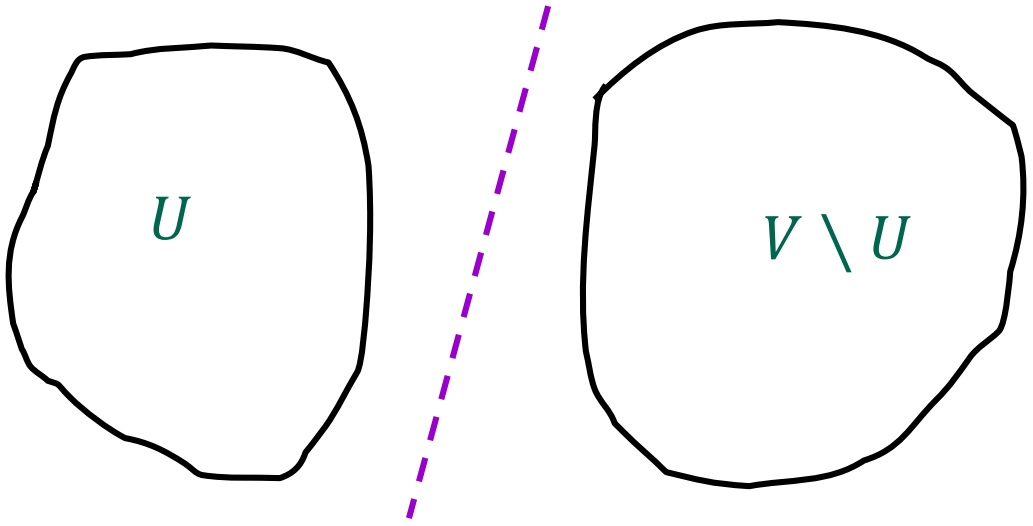


- Boruvka 1926
- Kruskal 1956
- Prim 1957 also by Jarnik 1930
- Karger, Klein, Tarjan 1995
  - Randomized expected linear time algorithm.
- Linear-time algorithms in special cases (dense graphs, integer weights)
- Chazelle (soft heap), 2000.
- Parallel algorithm in O(log n).

Cut Definition

Definitions:

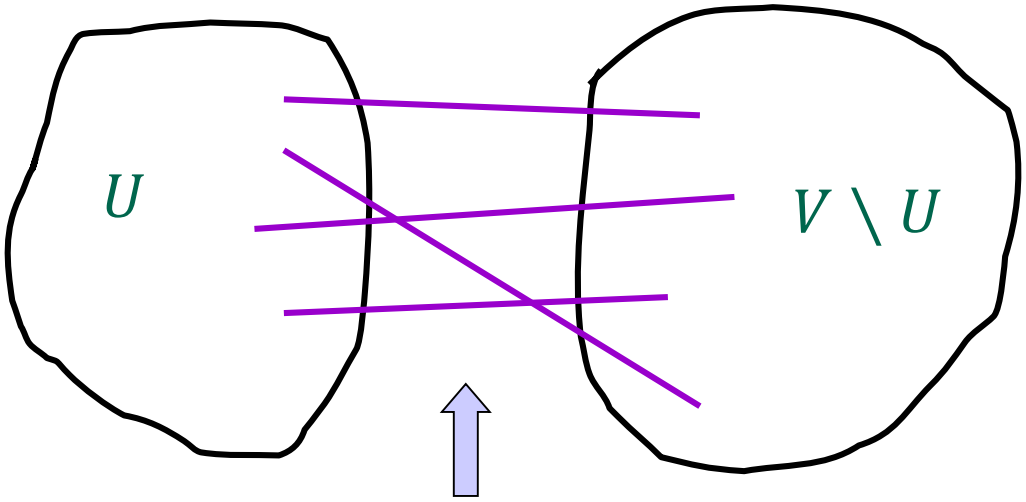
- A cut is a partition of the vertices into two disjoint subsets U and $V \setminus U$. We say that the cut is induced by U.



The cut induced by U

Cut Definition

- For a given cut U, the cut-edges are the edges that have one endpoint in U and one endpoint in $V \setminus U$.



The cut-edges of the cut induced by U

An Algorithm for MST

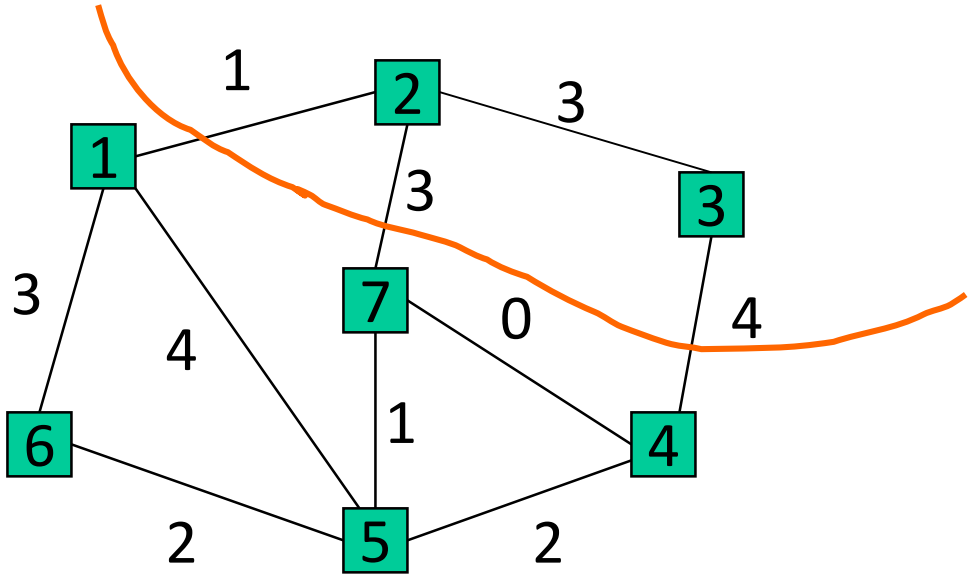
- The algorithm colors the edges of the graph. Initially, all edges are black.
- A blue edge – belongs to T.
- A red edge – does not belong to T.
- We continue to color edges until we have n-1 blue edges.
- How do we select which edge to color next? How do we select its color?

The Blue/Red Edge-coloring Rules

- The blue rule: Find a cut with no crossing blue edge. Color blue the cheapest black edge in the cut.
- The red rule: Find a cycle with no red edge. Color red the most expensive black edge in the cycle.

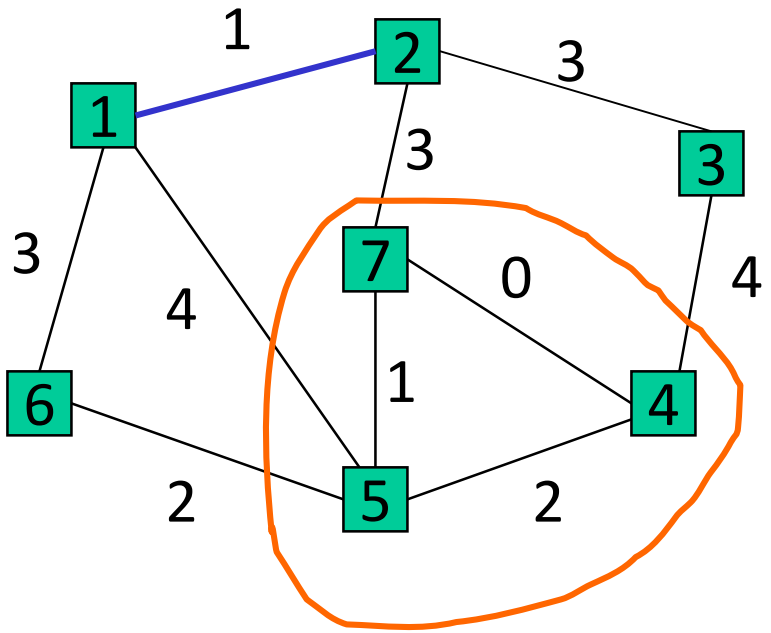
- These rules can be applied in any order.
- We will see two specific algorithms.

Example of Blue/Red rules (1)



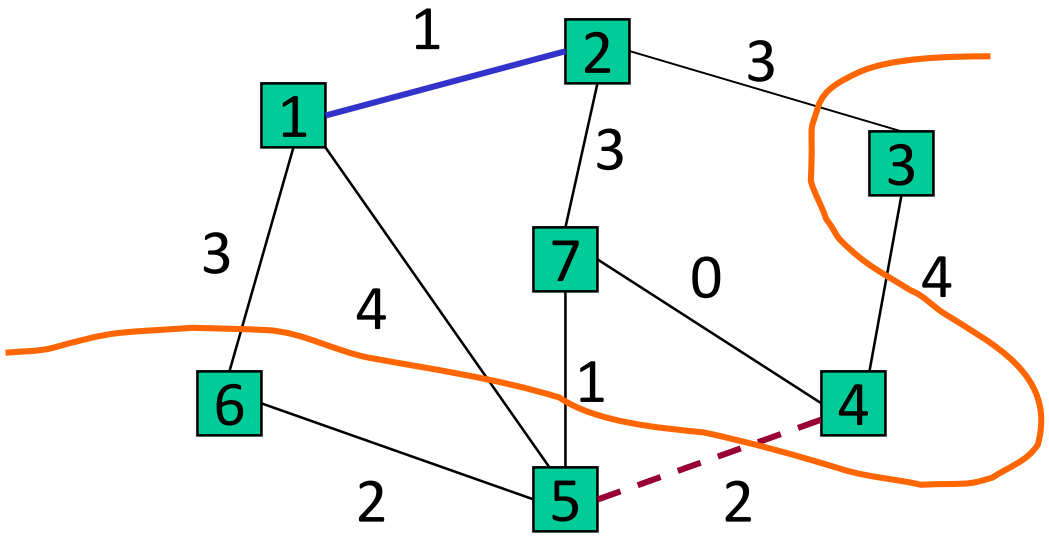
Consider the cut defined by {2,3}
- color (1,2) blue

Example of Blue/Red rules (2)



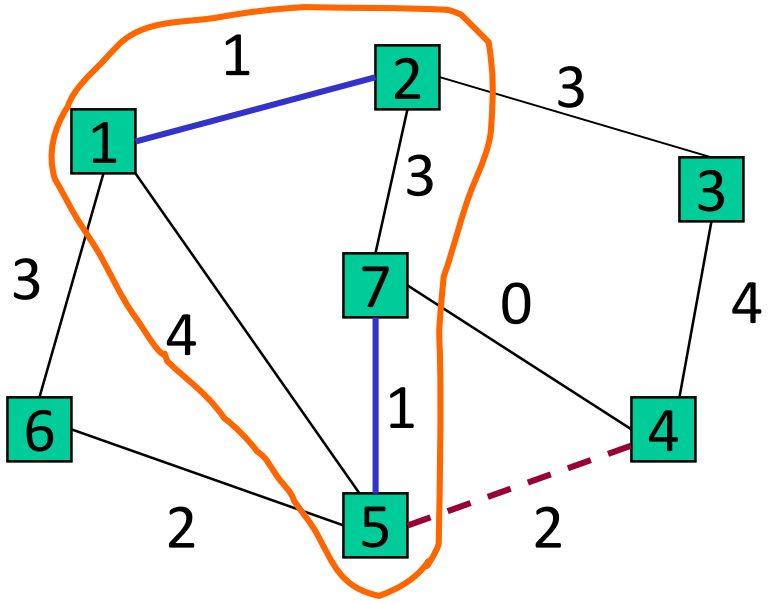
Consider the cycle (7-5-4)
- color (4,5) red

Example of Blue/Red rules (3)



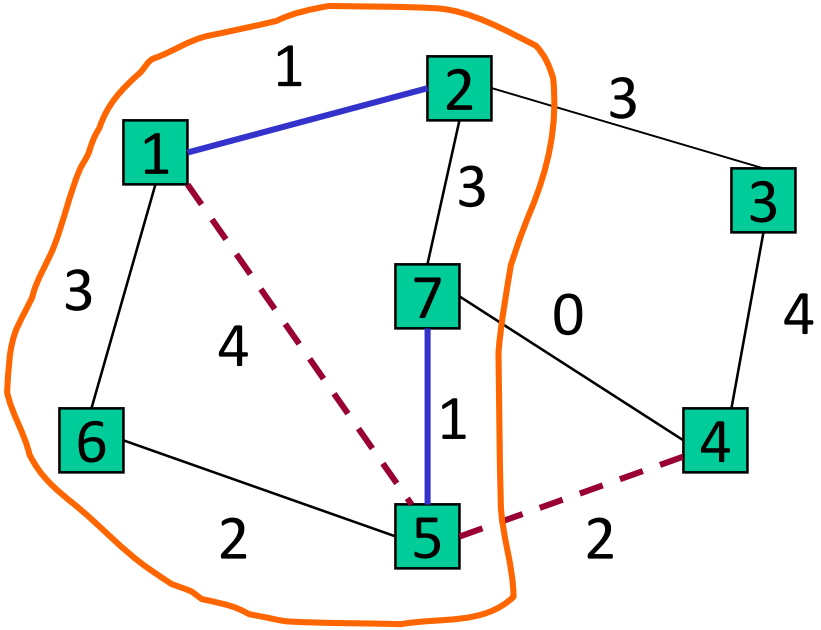
Consider the cut defined by {3,5,6}
- color (5,7) blue

Example of Blue/Red rules (4)



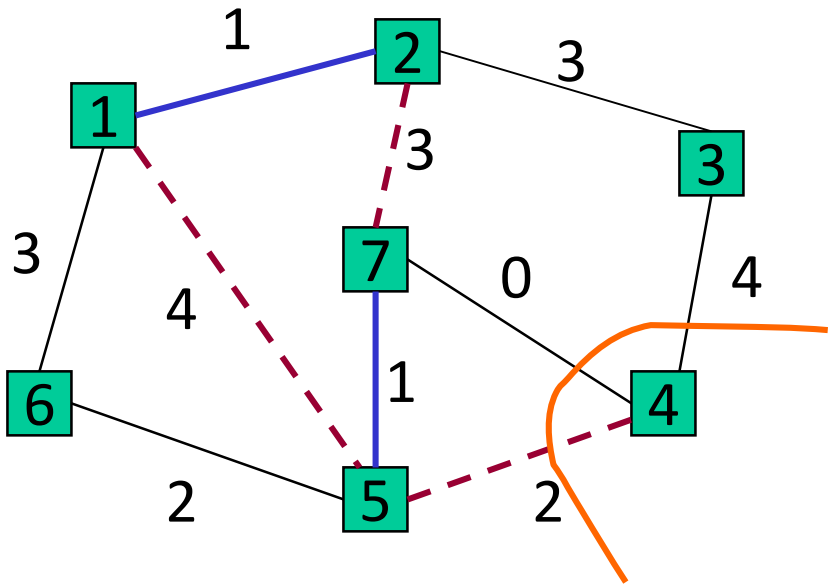
Consider the cycle (1-2-7-5)
- color (1,5) red

Example of Blue/Red rules (5)



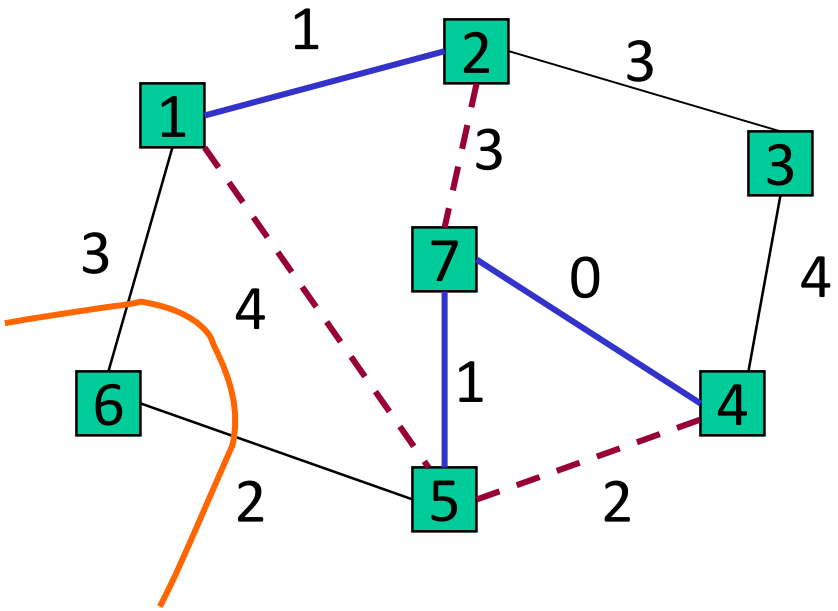
Consider the cycle (1-2-7-5-6)
- color (2,7) red.

Example of Blue/Red rules (6)



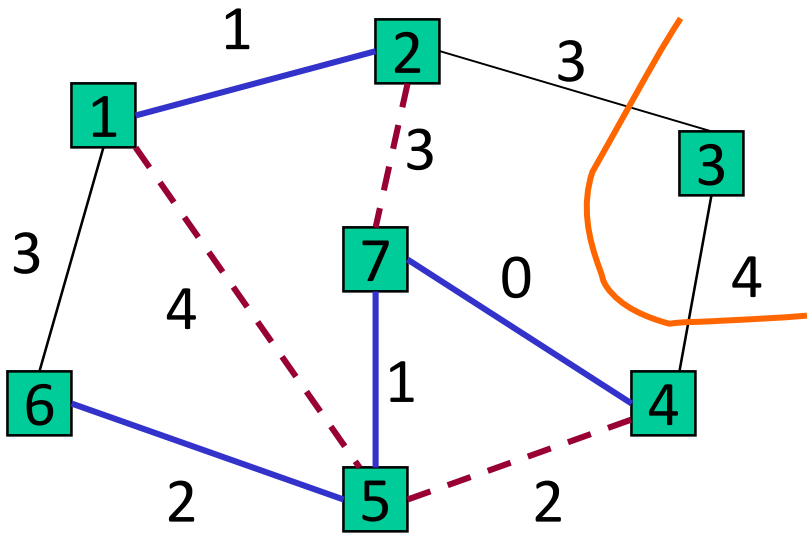
Consider the cut defined by {4}
- color (4,7) blue

Example of Blue/Red rules (7)



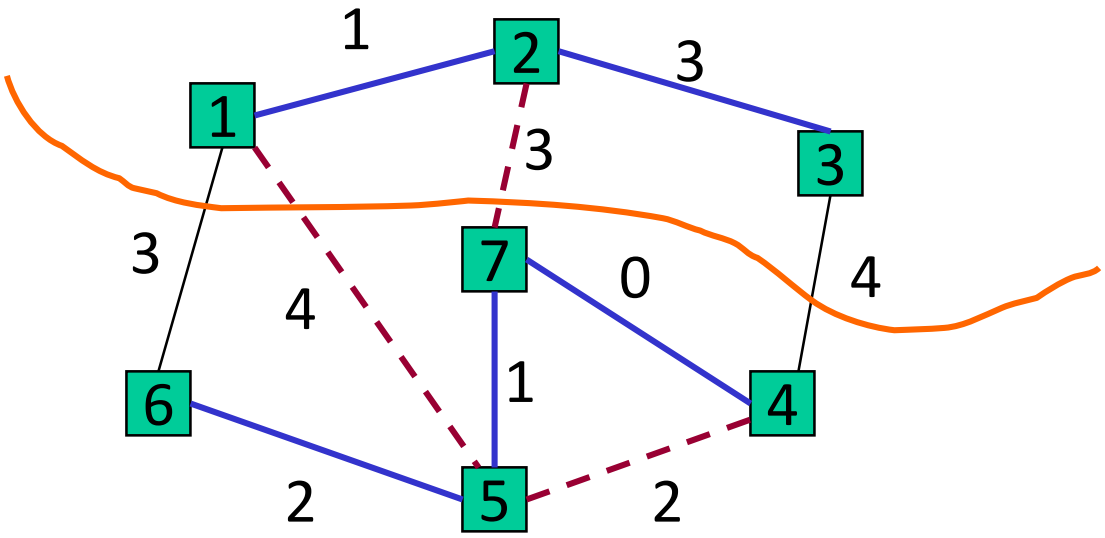
Consider the cut defined by {6}
- color (5,6) blue

Example of Blue/Red rules (8)



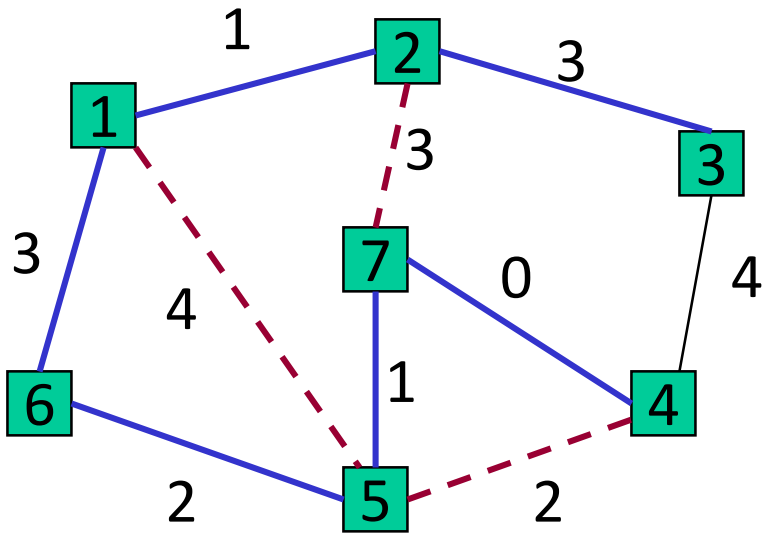
Consider the cut defined by {3}
- color (2,3) blue

Example of Blue/Red rules (9)



Consider the cut defined by {1,2,3}
- color (1,6) blue

Example of Blue/Red rules (10)



Final MST

Proof of Blue/Red Rules

Claim: for any $k \geq 0$, after coloring k edges, there exists an MST that includes all the blue edges and none of the red edges.

Proof: By induction on k.

- Base: $k=0$ trivially holds.
- Step: Assume the claim is true after coloring $k-1$ edges $e_1, e_2, \ldots, e_{k-1}$. Consider the coloring of e_k.
- Let $e_k=(u,v)$.

Note: The induction is based on a greedy choice property, but we do not prove it separately for the first step of the algorithm.

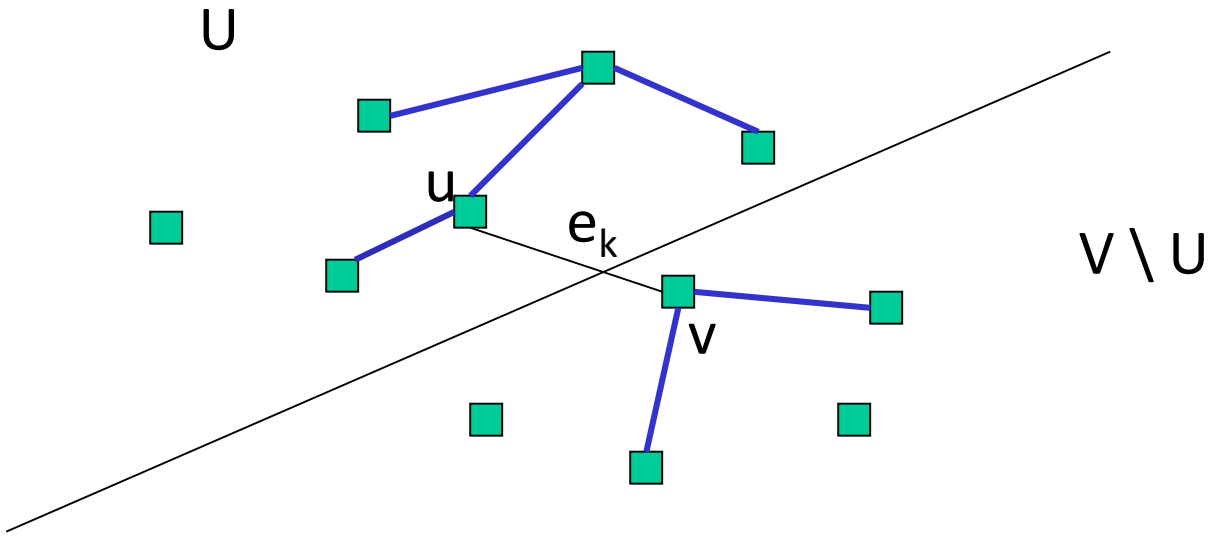
Case 1: Applying the Blue Rule

$e_k = (u,v)$ is colored blue.
Let T be an MST that includes the blue edges in $e_1, e_2, \ldots, e_{k-1}$ (such an MST exists by the induction hypothesis).
Assume that e_k is not in T.
Since T is a ST, it includes a path connecting u and v.
e_k was colored blue since it is the cheapest in some cut.
The path connecting u and v crosses this cut in some edge (x,y). It holds that $w(e_k) \leq w(x,y)$.

Consider $T' = T - \{(x,y)\} \cup \{e_k\}$.
T' is a ST whose cost is not larger than the cost of T. Thus, there exists an MST that includes e_k.

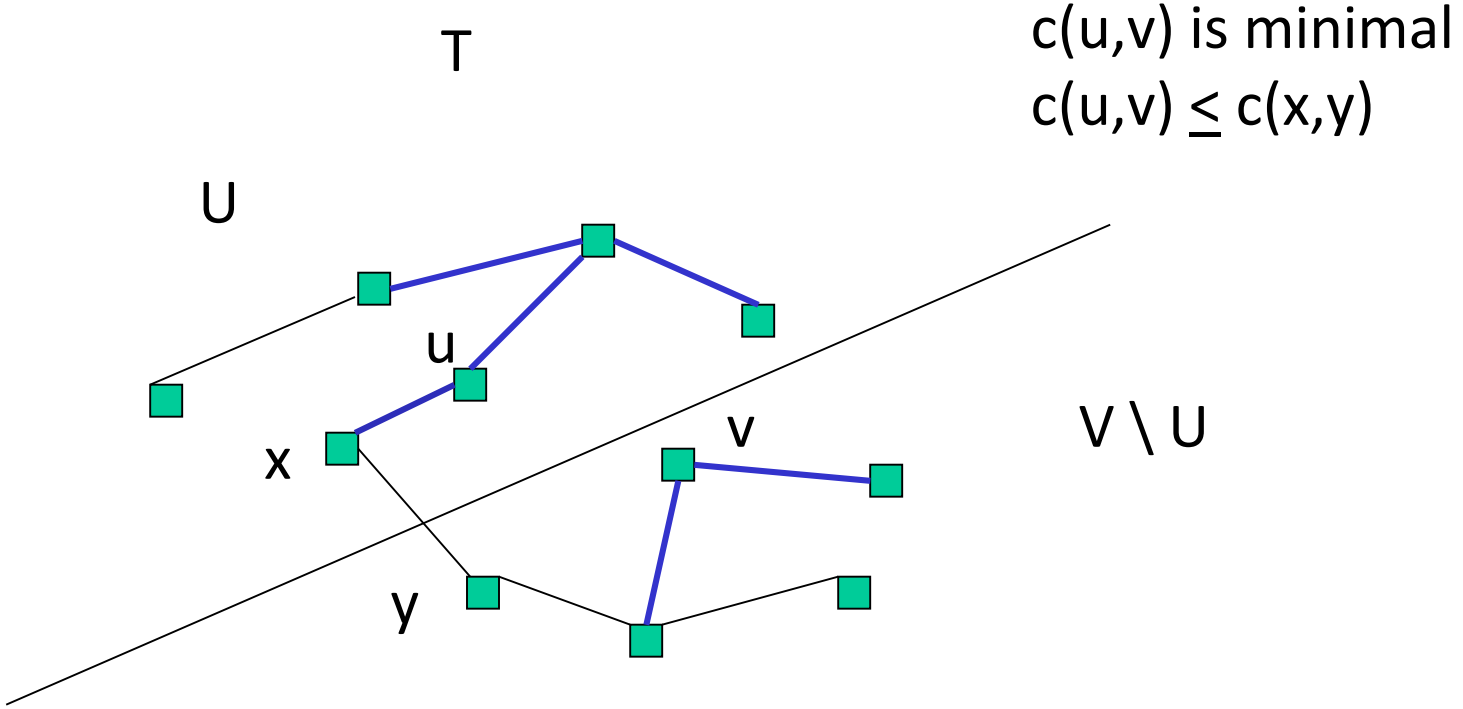
Case 1: Applying the Blue Rule

c(u,v) is minimal in some cut



Let T be an MST that agrees with the algorithm on the coloring of the first k-1 edges.

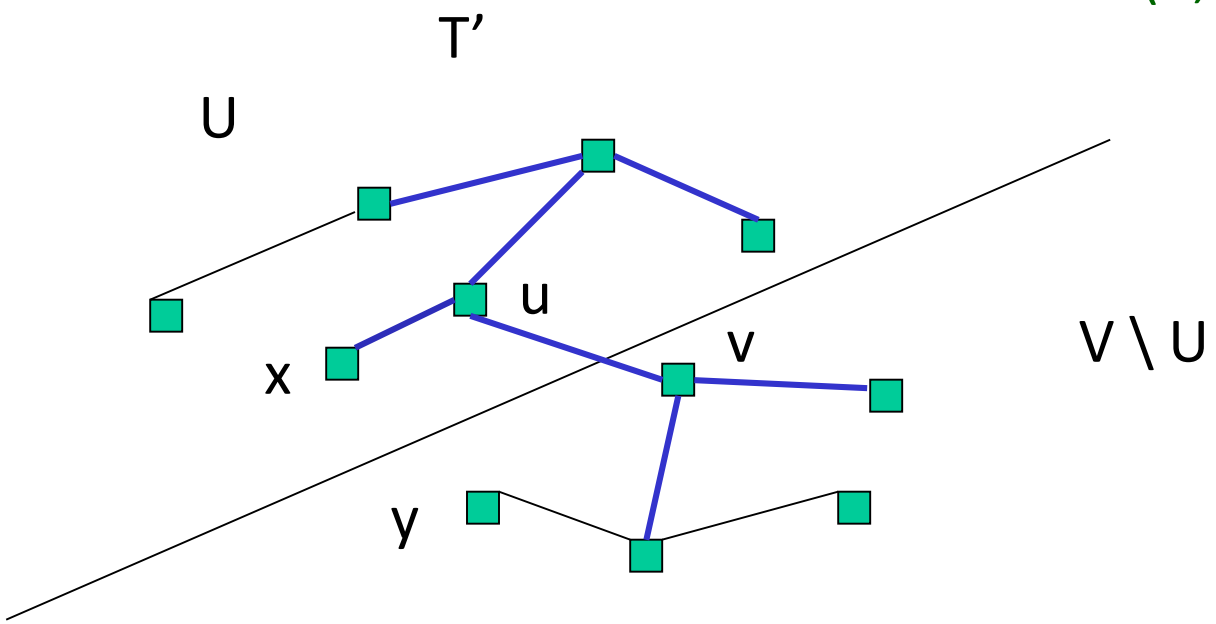
Case 1: Applying the Blue Rule



If $(u,v) \notin T$, then T must include some other edge (x,y) crossing the cut defined by U (T is connected, so there is a path connecting u and v).

Case 1: Applying the Blue Rule

Consider T’ = T ∪ (u,v) -(x,y)



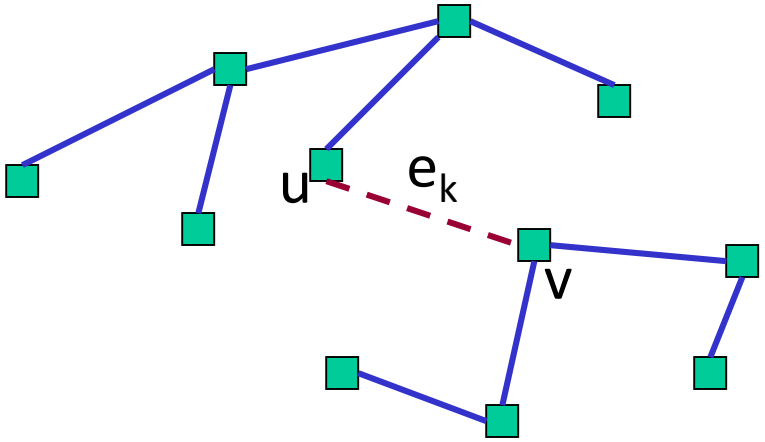
c(T’) = c(T) + c(u,v) - c(x,y)
c(T’) ≤ c(T)

T’ is also a minimum spanning tree, and it includes e_k

Case 2: Applying the Red Rule

e_k is colored red.
Let T be an MST that includes the blue edges in $e_1, e_2, \ldots, e_{k-1}$ (such an MST exists by the induction hypothesis).
Assume it includes also e_k. Remove e_k from T. The result is two trees T_1, T_2. The cycle on which the red rule was applied includes another edge e that connects T_1 and T_2.
This edge is not blue, as otherwise, together with e_k, T includes a cycle. e_k was colored red, therefore $w(e) \leq w(e_k)$.
Consider $T' = T - \{e_k\} \cup \{e\}$. T' is a ST whose cost is not larger than the cost of T. Thus, there exists an MST that does not include e_k.

Case 2: Applying the Red Rule

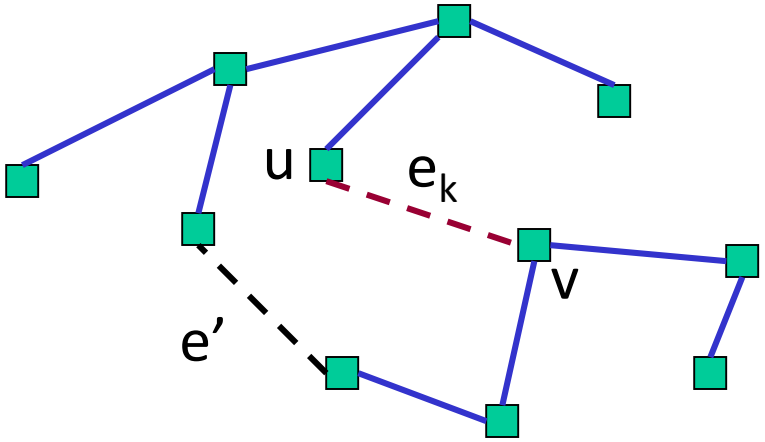


c(u,v) is maximal in some cycle

Assume $(u,v) \in T$.

By removing (u,v) from T we get two components.

Case 2: Applying the Red Rule



c(u,v) is maximal in some cycle

The cycle that causes us to color (u,v) red includes an edge connecting the two components (whose cost is at most c(u,v).

Consider $T' = T - \{e_k\} \cup \{e'\}$. $c(T') \leq c(T)$

➔ There is an MST, that does not include e_k

הפעלת הכללים הכחול והאדום- הוכחה

טענה: לכל $k \geq 0$, אחרי צביעת k קשתות, קיים עץ פורש מינימום שמכיל את כל הקשתות הכחולות ולא כולל אף קשת אדומה.

הוכחה: אינדוקציה על k.

בסיס: $k=0$ נכון באופן ריק

צעד: נניח שנכון אחרי שצבענו את $e_1, e_2, \ldots, e_{k-1}$ ונתייחס לצביעה של $e_k=(u,v)$.

Note: The induction is based on a greedy choice property, but we do not prove it separately for the first step of the algorithm

מקרה 1: e_k נצבעה כחול

נניח ש-e_k נצבעה כחול. יהי T עפ"מ שמכיל את כל הקשתות הכחולות מתוך $e_1,e_2,\ldots,e_{k-1}$ (קיים כזה, עפ"י הנחת האינד').

נניח שאינו כולל את e_k.

מכיוון ש- T עץ פורש, יש בו מסלול בין u ל-v.

e_k נצבעה בכחול משום שהיא הזולה ביותר בחתך מסוים.

המסלול שקיים ב-T בין u ל-v חותך את החתך, נניח שבקשת (x,y).

על פי בחירת e_k, בהכרח מתקיים ש- $w(e_k)\leq w(x,y)$.

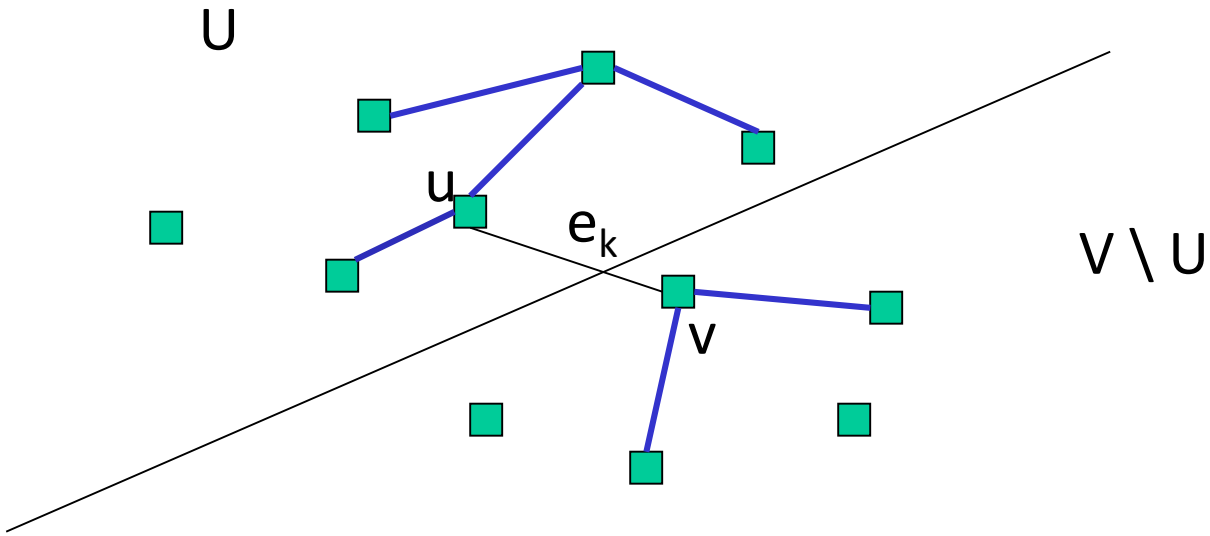
יהי $T'=T-\{(x,y)\}\cup\{e_k\}$.

T' הוא עץ פורש שמשקלו לא גבוה ממשקל T.

לבן קיים עפ"מ שכולל גם את e_k.

מקרה 1: e_k נצבעה כחול

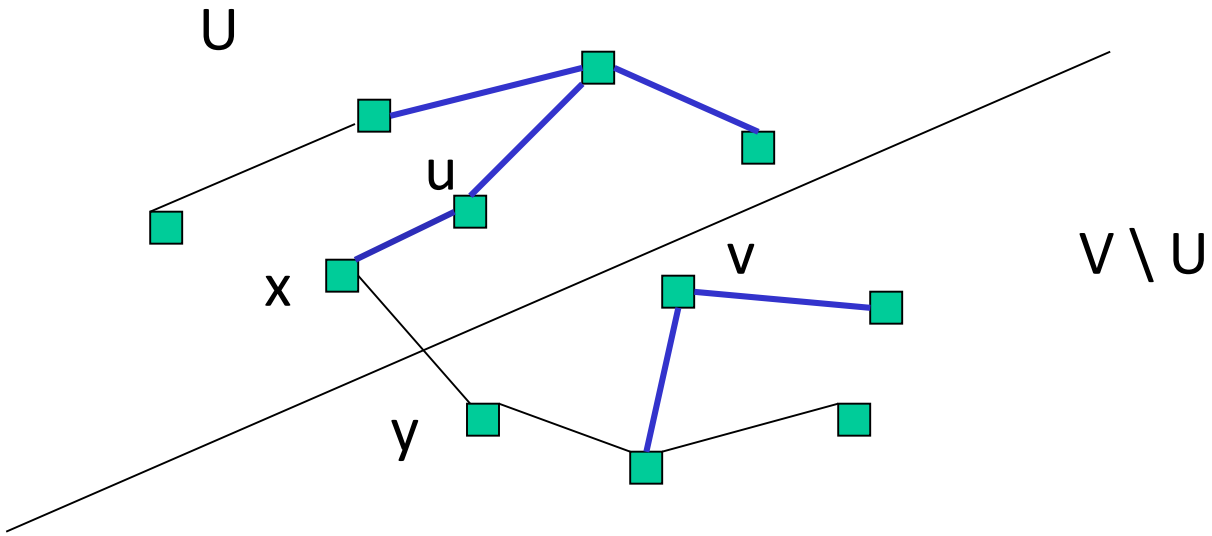
יהי T עפ"מ שמסכים עם האלגוריתם על צביעת k-1 הקשתות הראשונות



$e_k=(u,v)$ נצבעה בכחול מכיוון ש- $c(u,v)$ מינימלי באיזשהו חתך

מקרה 1: e_k נצבעה כחול

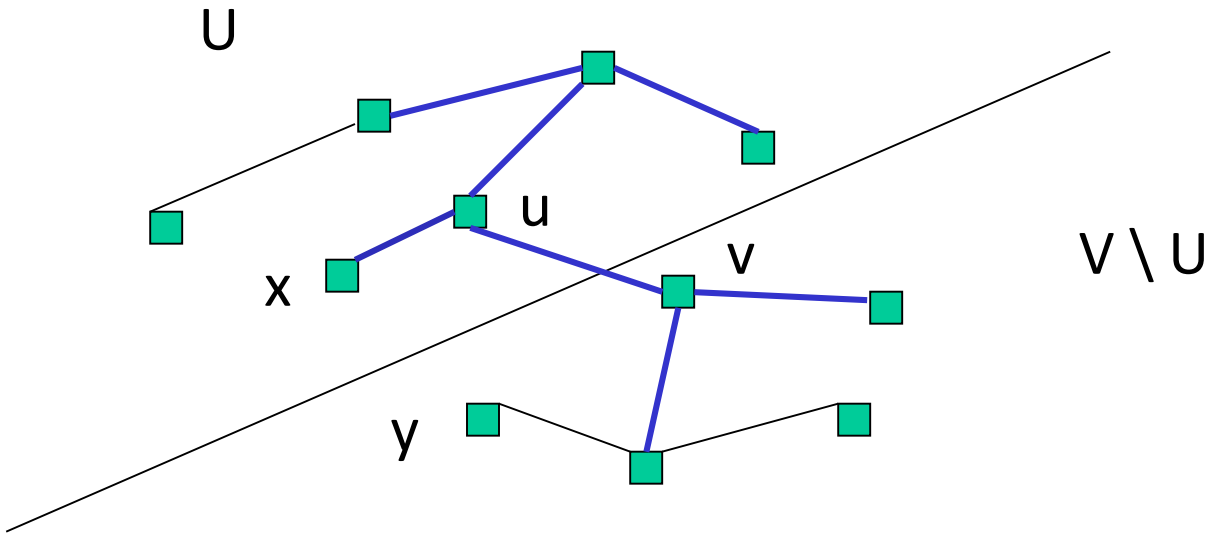
אם $(u,v) \notin T$, אז המסלול שקיים ב-T בין u ל-v כולל קשת אחרת בחתך, נניח שאת הקשת (x,y).



c(u,v) מינימלי בחתך, לכן $c(u,v) \leq c(x,y)$

מקרה 1: e_k נצבעה כחול

יהי $T' = T \cup (u,v) - (x,y)$



$c(T') = c(T) + c(u,v) - c(x,y)$

$c(T') \leq c(T)$

'T הוא עץ פורש שמשקלו לא גבוה ממשקל T.

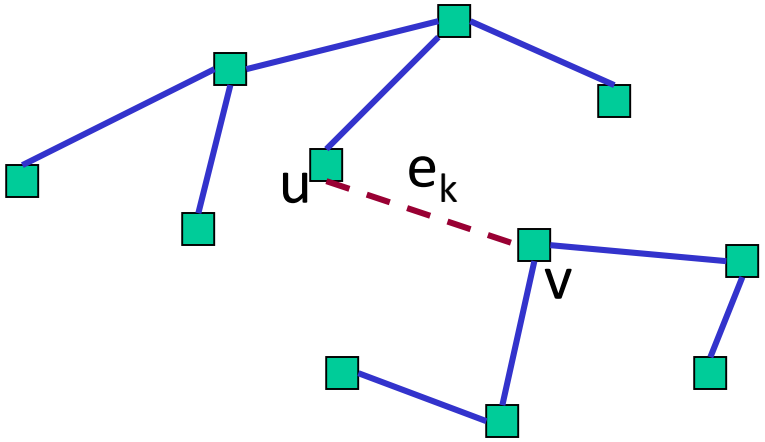
לפן קיים עפ"מ שכולל את e_k.

מקרה 2: e_k נצבעה אדום

e_k נצבעה אדום. יהי T עפ"מ שמכיל את כל הקשתות הכחולות מתוך $e_1,e_2,\ldots,e_{k-1}$ (קיים כזה, עפ"י הנחת האינד'). נניח שהוא כולל את e_k.
נוריד את e_k מ-T. נקבל שני רכיבים T_1,T_2.
המעגל עליו הופעל הכלל האדום שבו e_k נצבעה, כולל עוד קשת e שמחברת בין T_1 ל-T_2. קשת זו לא היתה כחולה, כי אז יחד עם e_k היה ב-T מעגל.
e_k נצבעה באדום ולכן $w(e)\leq w(e_k)$.
יהי $T'=T-\{e_k\}\cup\{e\}$. T' הוא עץ פורש שמשקלו לא גבוה ממשקל T. לכן קיים עפ"מ שלא כולל את e_k.

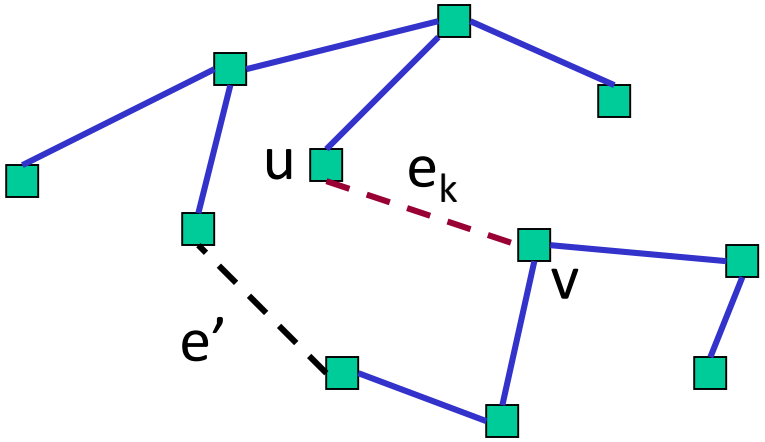
מקרה 2: e_k נצבעה אדום

c(u,v) מקסימלי
באיזשהו מעגל



יהי T עפ"מ שמכיל את כל הקשתות הכחולות מתוך $e_1,e_2,\ldots,e_{k-1}$ (קיים בזה, עפ"י הנחת האינד'). נניח שהוא כולל את e_k. נוריד את e_k מ-T. נקבל שני רכיבים T_1,T_2.

מקרה 2: e_k נצבעה אדום



המעגל עליו הופעל הכלל האדום שבו e_k נצבעה, כולל עוד קשת e שמחברת בין T_1 ל-T_2. e_k היא היקרה ביותר במעגל ולכן $w(e)\leq w(e_k)$.

יהי $T'=T-\{e_k\}\cup\{e\}$. T' הוא עץ פורש שמשקלו לא גבוה ממשקל T. לכן קיים עפ"מ שלא כולל את e_k.

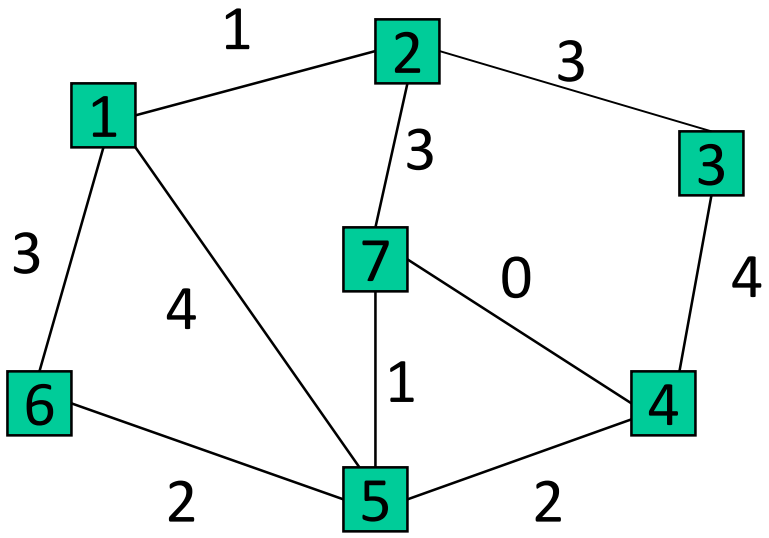
Kruskal's Greedy Algorithm

```
Sort the edges by increasing cost;
Initialize T to be empty;
For each edge e chosen in increasing order do
   if adding e does not form a cycle then
      add e to T
```

Proof: The algorithm follows the blue/red rules:

- If e closes a cycle – apply the red rule (by the sorting, e is the most expensive in this cycle).
- Otherwise – apply the blue rule (e connects two components, consider the cut defined by any of them. e is the cheapest edge in this cut).

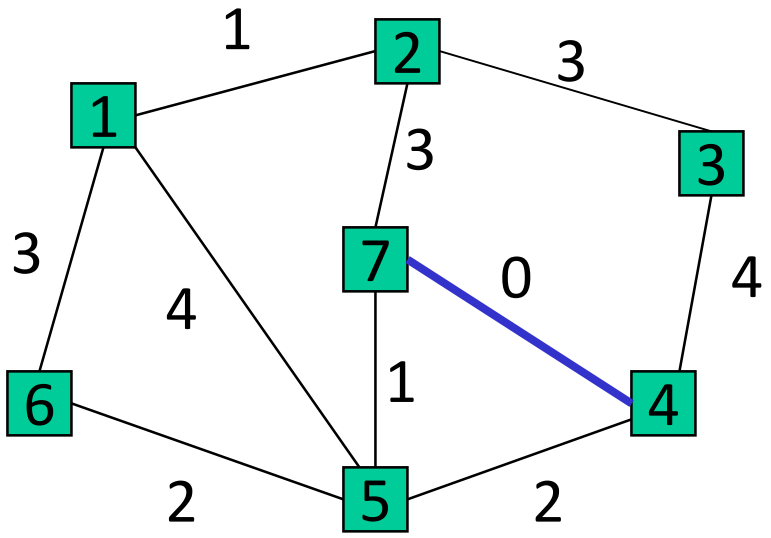
Example of Kruskal 1



{7,4} {2,1} {7,5} {5,6} {5,4} {1,6} {2,7} {2,3} {3,4} {1,5}

0 1 1 2 2 3 3 3 4 4

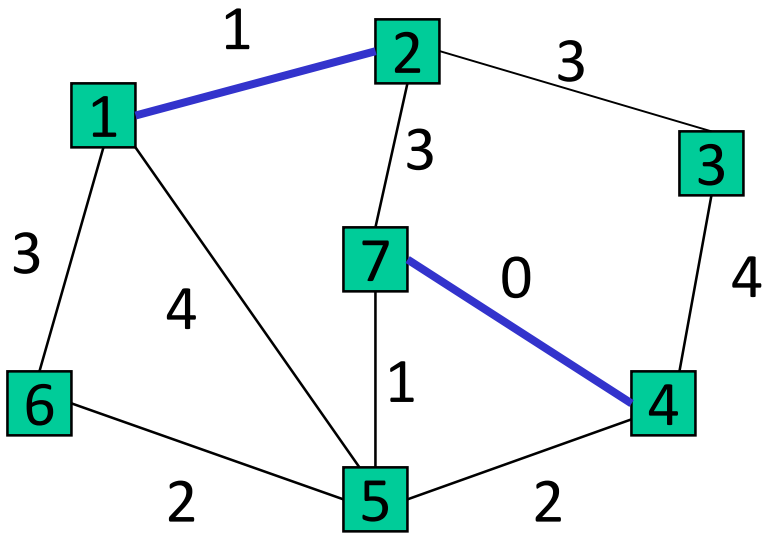
Example of Kruskal 2



~~{7,4}~~ {2,1} {7,5} {5,6} {5,4} {1,6} {2,7} {2,3} {3,4} {1,5}

~~0~~ 1 1 2 2 3 3 3 4 4

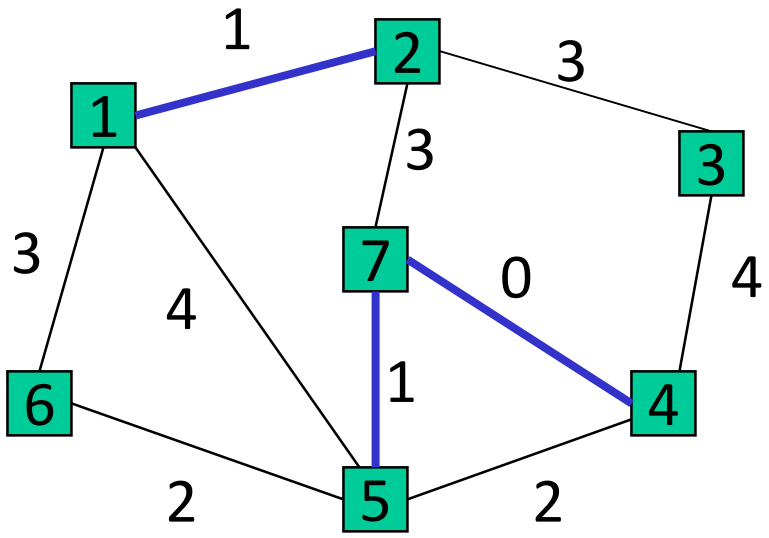
Example of Kruskal 2



~~{7,4}~~ ~~{2,1}~~ {7,5} {5,6} {5,4} {1,6} {2,7} {2,3} {3,4} {1,5}

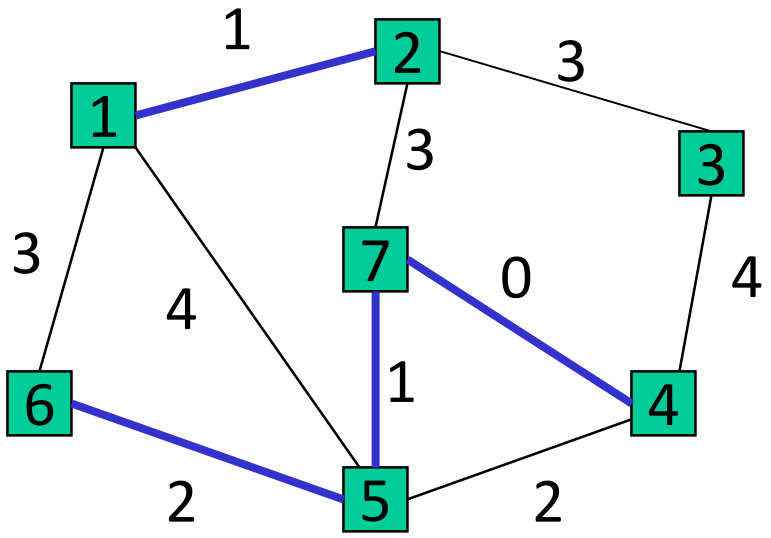
~~0~~ ~~1~~ 1 2 2 3 3 3 4 4

Example of Kruskal 3



{7,4}	{2,1}	{7,5}	{5,6}	{5,4}	{1,6}	{2,7}	{2,3}	{3,4}	{1,5}
0	1	1	2	2	3	3	3	4	4

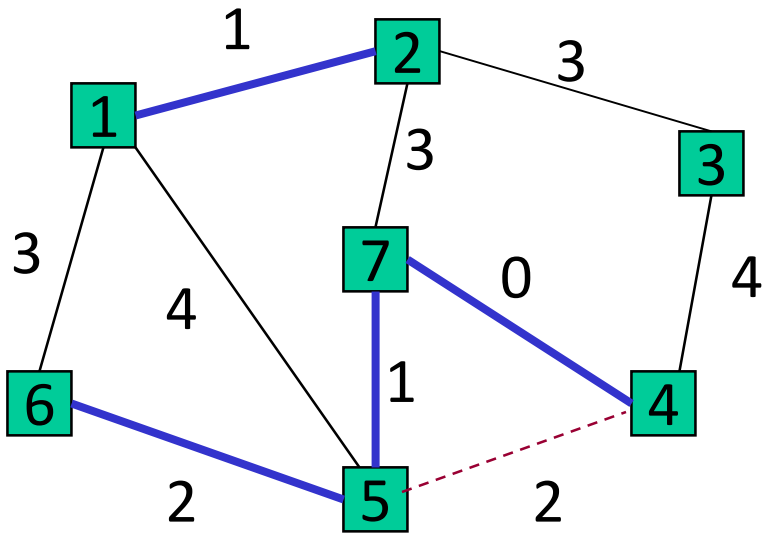
Example of Kruskal 4



~~{7,4}~~ ~~{2,1}~~ ~~{7,5}~~ ~~{5,6}~~ {5,4} {1,6} {2,7} {2,3} {3,4} {1,5}

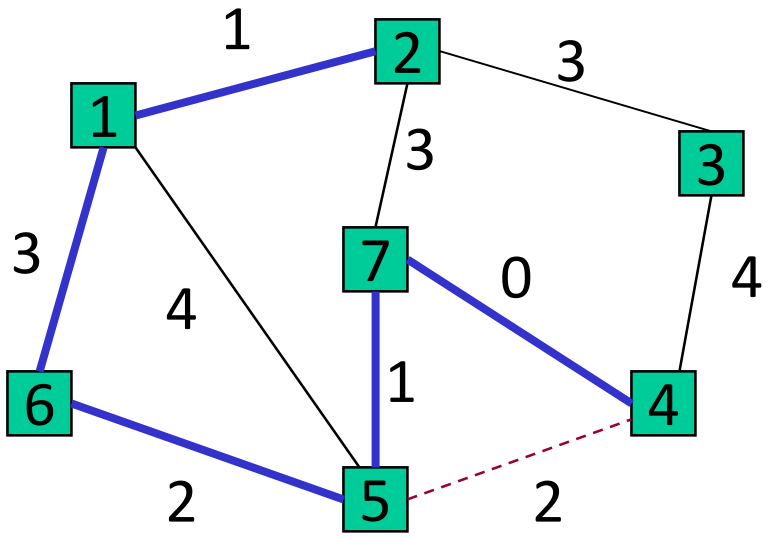
~~0~~ ~~1~~ ~~1~~ ~~2~~ 2 3 3 3 4 4

Example of Kruskal 5



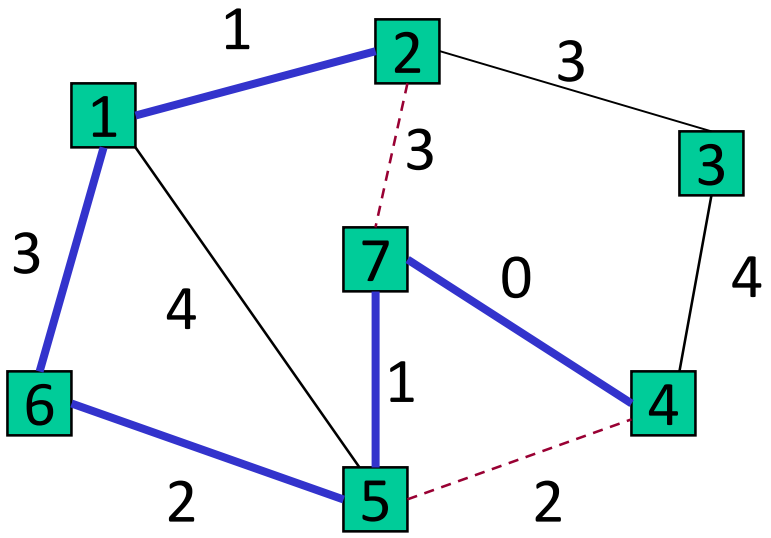
{7,4}	{2,1}	{7,5}	{5,6}	{5,4}	{1,6}	{2,7}	{2,3}	{3,4}	{1,5}
0	1	1	2	2	3	3	3	4	4

Example of Kruskal 6



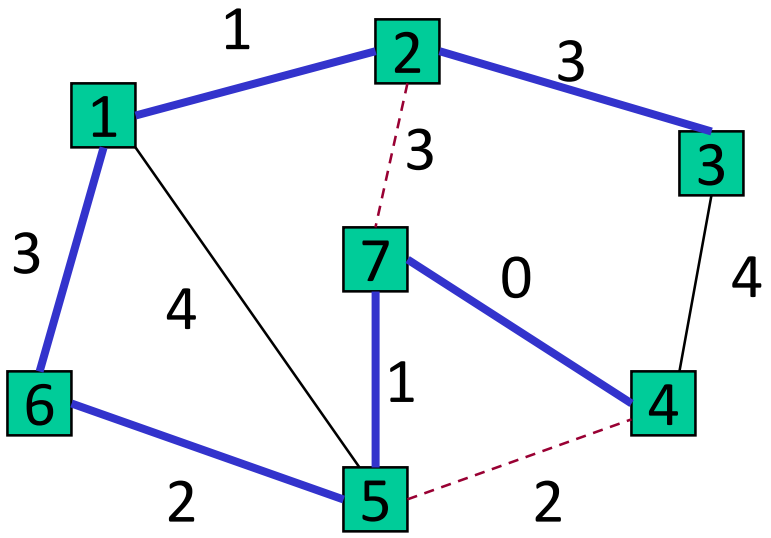
{7,4}	{2,1}	{7,5}	{5,6}	{5,4}	{1,6}	{2,7}	{2,3}	{3,4}	{1,5}
0	1	1	2	2	3	3	3	4	4

Example of Kruskal 7



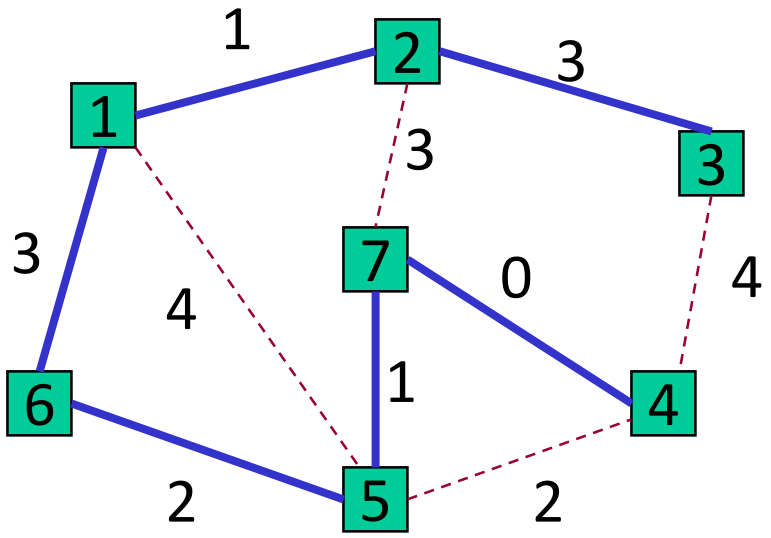
{7,4}	{2,1}	{7,5}	{5,6}	{5,4}	{1,6}	{2,7}	{2,3}	{3,4}	{1,5}
0	1	1	2	2	3	3	3	4	4

Example of Kruskal 8



{7,4}	{2,1}	{7,5}	{5,6}	{5,4}	{1,6}	{2,7}	{2,3}	{3,4}	{1,5}
0	1	1	2	2	3	3	3	4	4

Example of Kruskal 9,10



{7,4}	{2,1}	{7,5}	{5,6}	{5,4}	{1,6}	{2,7}	{2,3}	{3,4}	{1,5}
0	1	1	2	2	3	3	3	4	4

Implementation of Kruskal

- We need a sorted edge list

{7,4}	{2,1}	{7,5}	{5,6}	{5,4}	{1,6}	{2,7}	{2,3}	{3,4}	{1,5}
0	1	1	2	2	3	3	3	4	4

- We need to be able to check whether an edge closes a cycle.
- We will use the Disjoint Union / Find data structure.
  - Union(a,b) - union the disjoint sets named by a and b
  - Find(a) returns the name of the set containing a

Disjoint Union - Find

- Maintain a set of disjoint sets.
  - {3,5,7} , {4,2,8}, {9}, {1,6}
- Each set has a unique name, one of its members
  - {3,5,7} , {4,2,8}, {9}, {1,6}

Union

- Union(x,y) – take the union of two sets named x and y
  - {3,5,7} , {4,2,8}, {9}, {1,6}
  - Union(5,1)

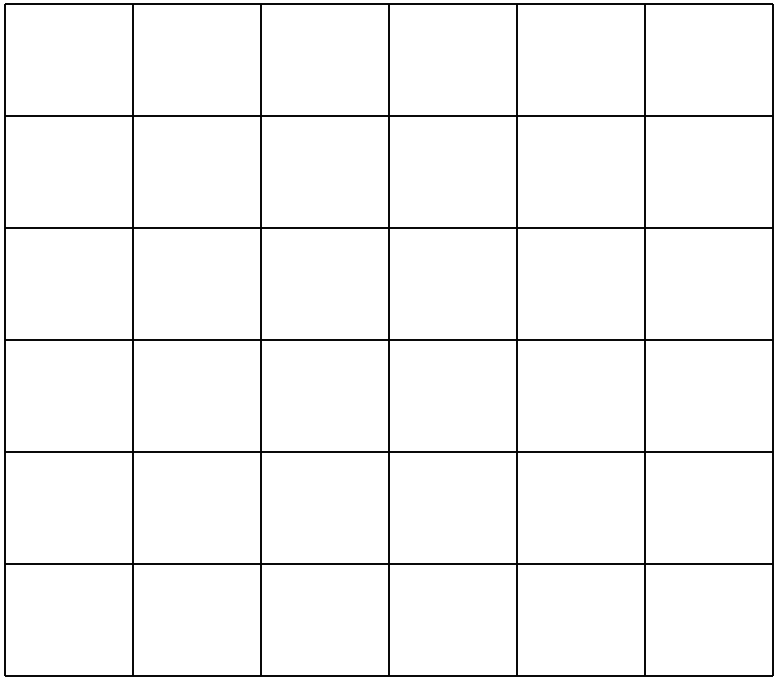
    {3,5,7,1,6}, {4,2,8}, {9},

Find

- Find(x) – return the name of the set containing x.
  - {3,5,7,1,6}, {4,2,8}, {9},
  - Find(1) = 5
  - Find(4) = 8

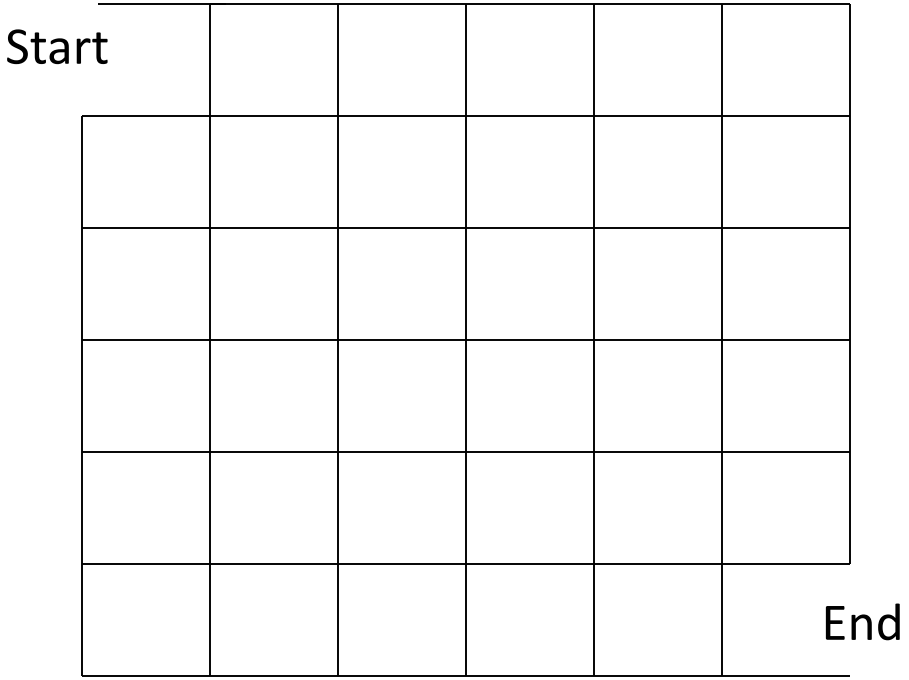
Warm Up: Maze construction

- Build a random maze by erasing edges.



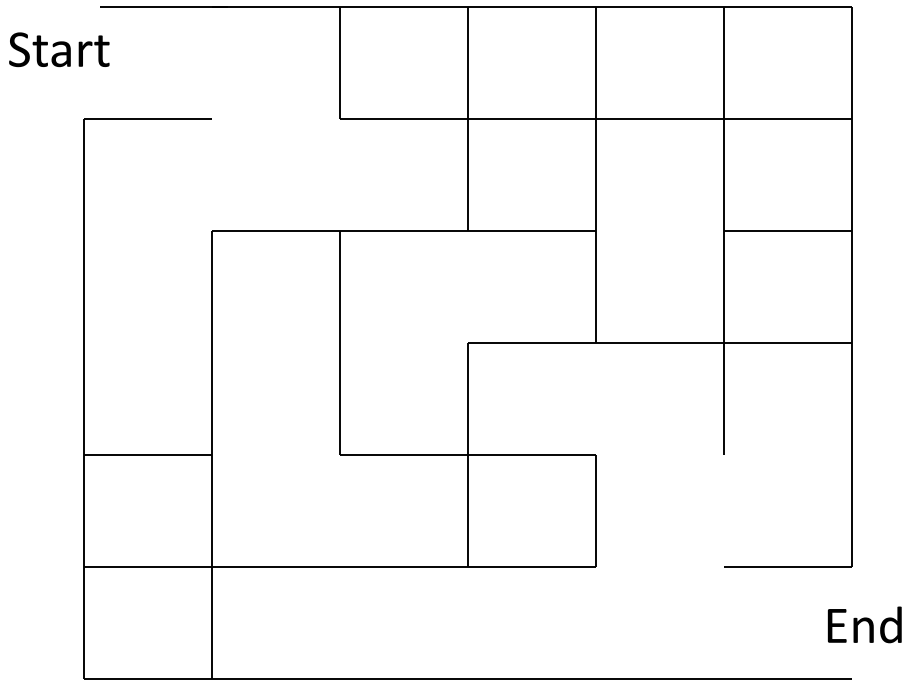
Maze construction

- Pick Start and End



Maze construction

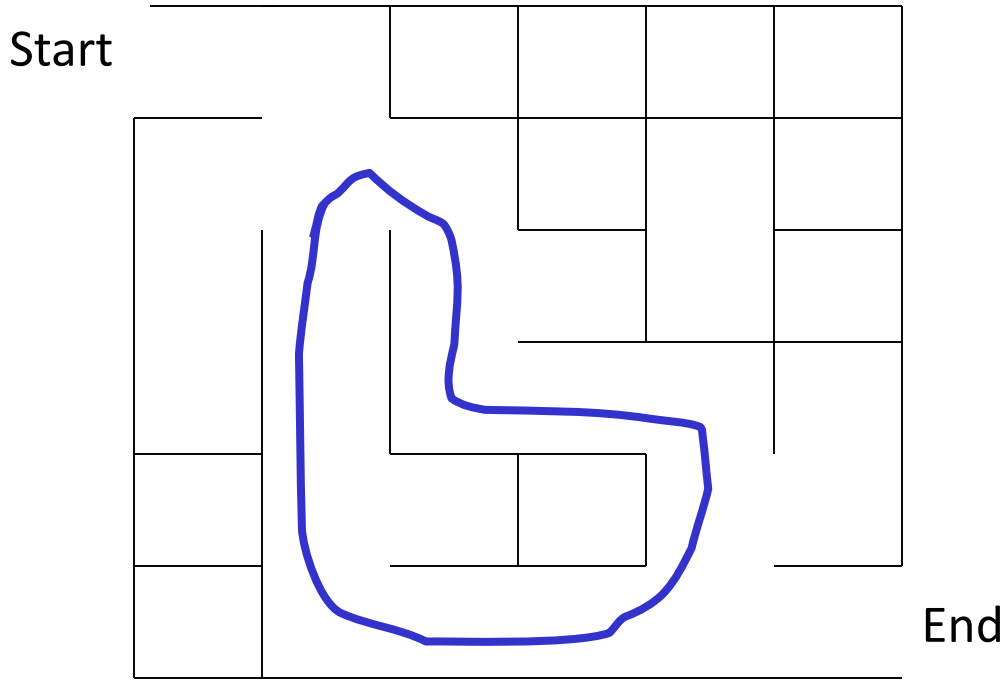
- Repeatedly pick random edges to delete.



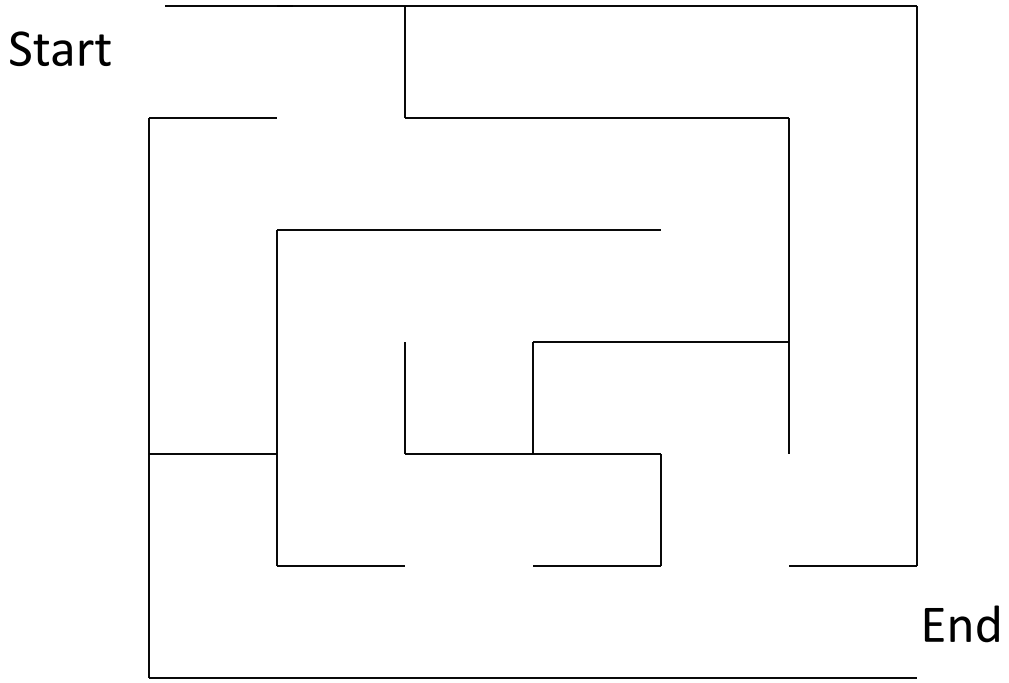
Desired Properties

- None of the boundary is deleted
- Every cell is reachable from every other cell.
- There are no cycles – no cell can reach itself by a path unless it retraces some part of the path.

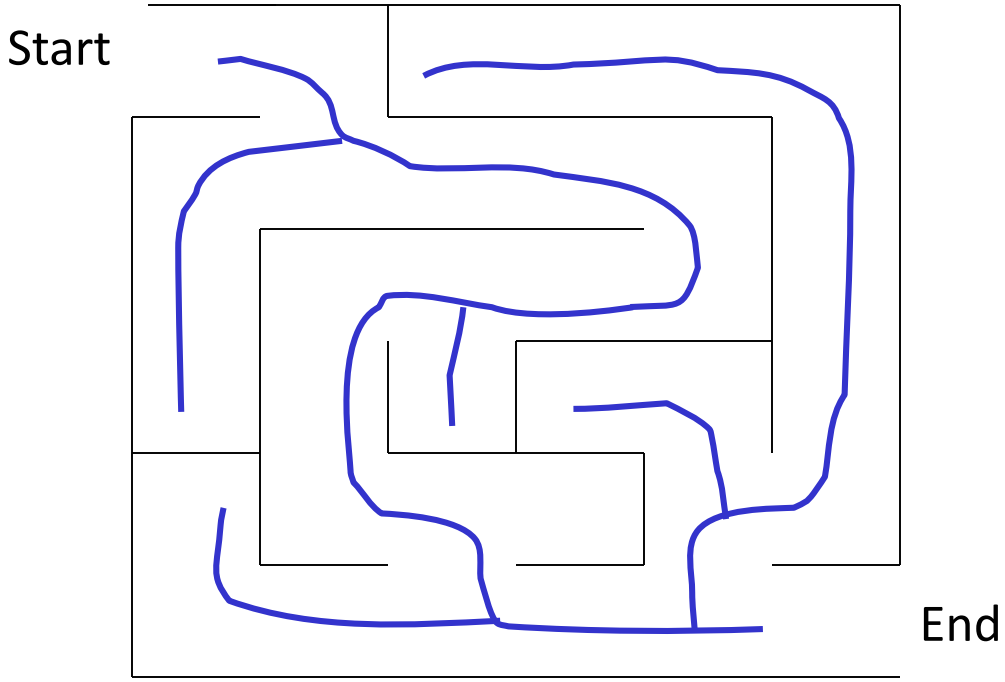
A Cycle (we don’t want that)



A Good Solution



Good Solution : A Hidden Tree



Number the Cells

Start with disjoint sets S ={ {1}, {2}, {3}, {4},… {36} } one set per cell. We have all possible walls E ={ (1,2), (1,7), (2,8), (2,3), … } 60 walls total.

Start

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36

End

Maze construction Algorithm

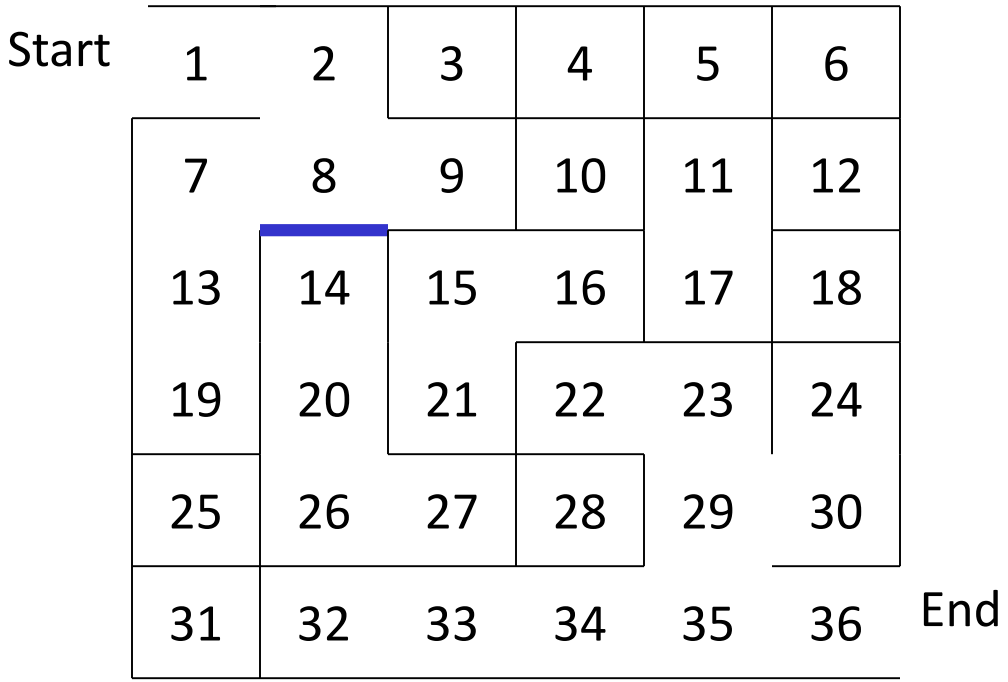
- S = set of sets of connected cells
- E = set of walls

```
E' ← a random shuffle of E
While there is more than one set in S:
   (x,y) ← pick next wall in E'
   u ← Find(x);  v ← Find(y);
   if u ≠ v then:
        Remove (x,y) from E
        Union(u,v)  // knock down the wall between
                    // the cells (cells in the same
                    // set are connected)
```

- If u=v there is already a path between x and y
- All remaining members of E form the maze

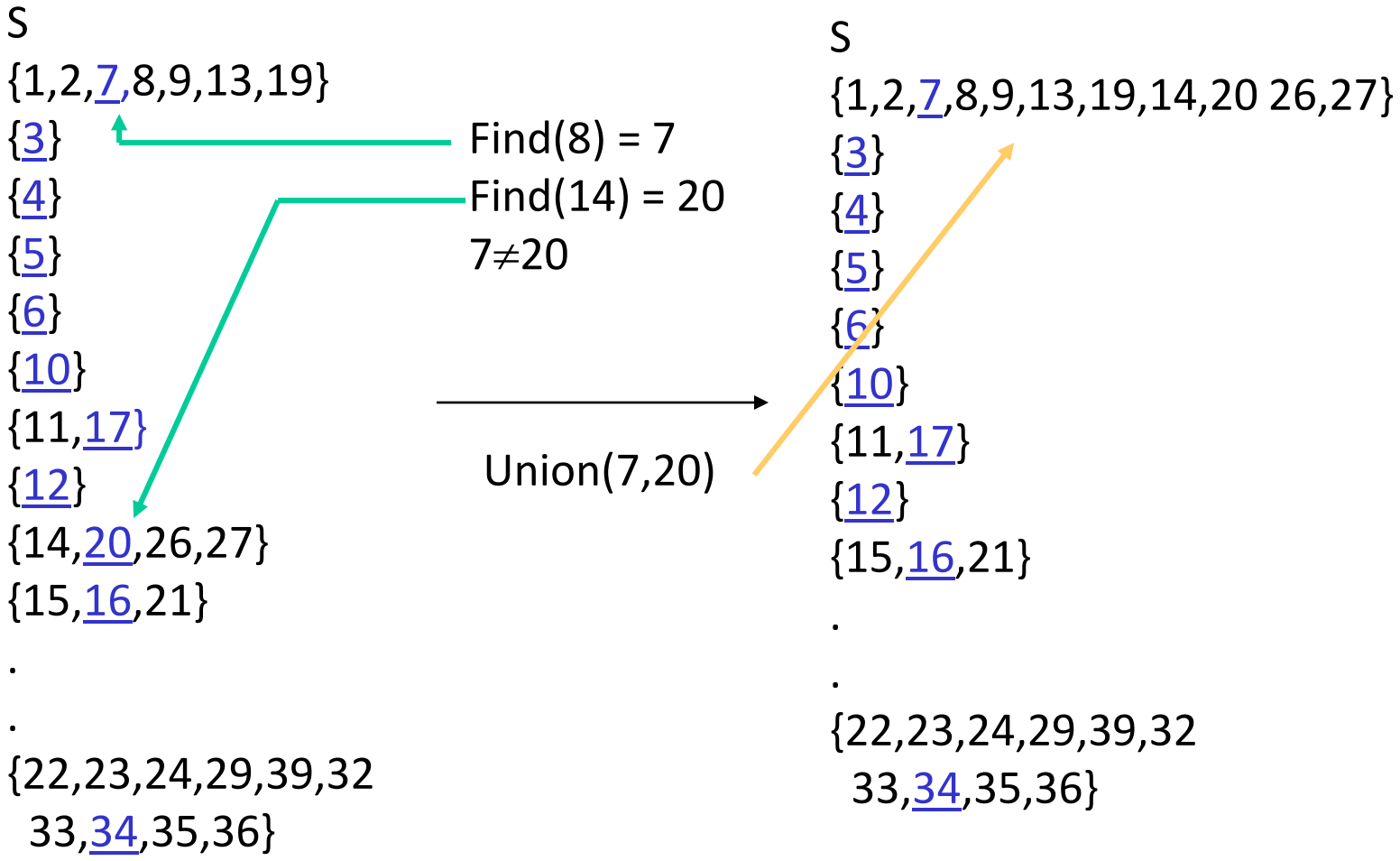
Example Step – removing a wall

Pick (8,14)



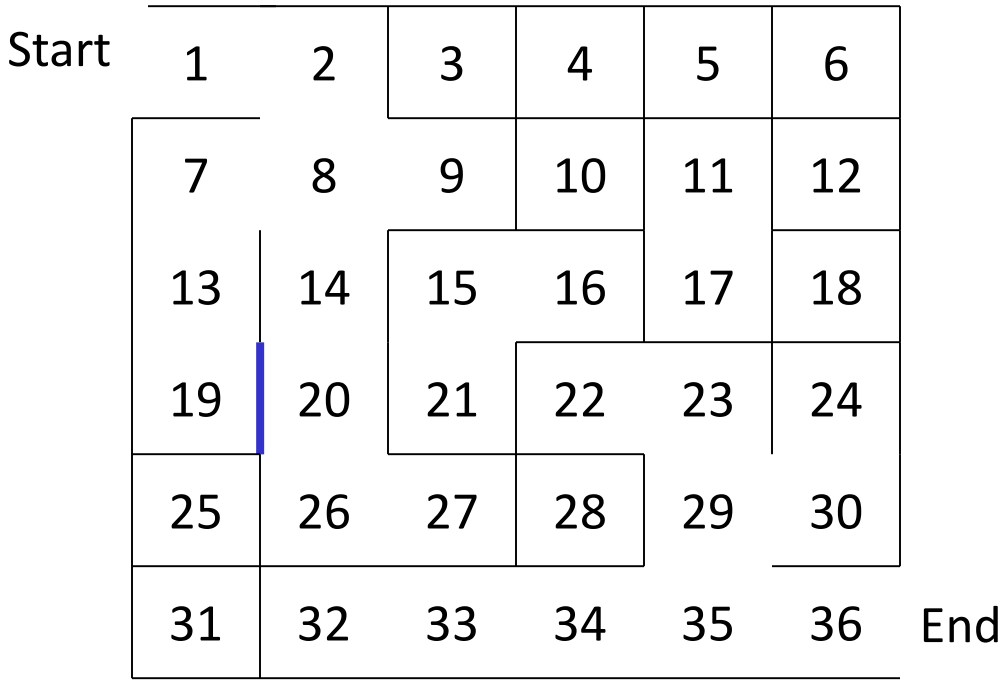
S
{1,2,7,8,9,13,19}
{3}
{4}
{5}
{6}
{10}
{11,17}
{12}
{14,20,26,27}
{15,16,21}
.
.
{22,23,24,29,30,32
33,34,35,36}

Example – removing a wall



Example – no removing

Pick (19,20)



S

{1,2,7,8,9,13,19
14,20,26,27}

{3}

{4}

{5}

{6}

{10}

{11,17}

{12}

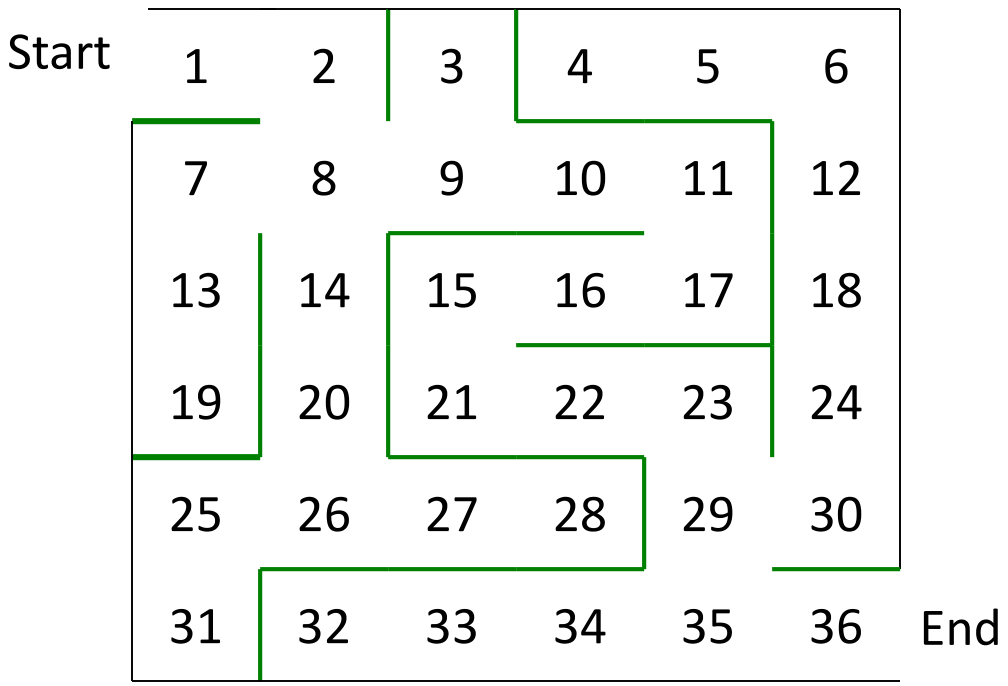
{15,16,21}

.

.

{22,23,24,29,39,32
33,34,35,36}

Example at the End

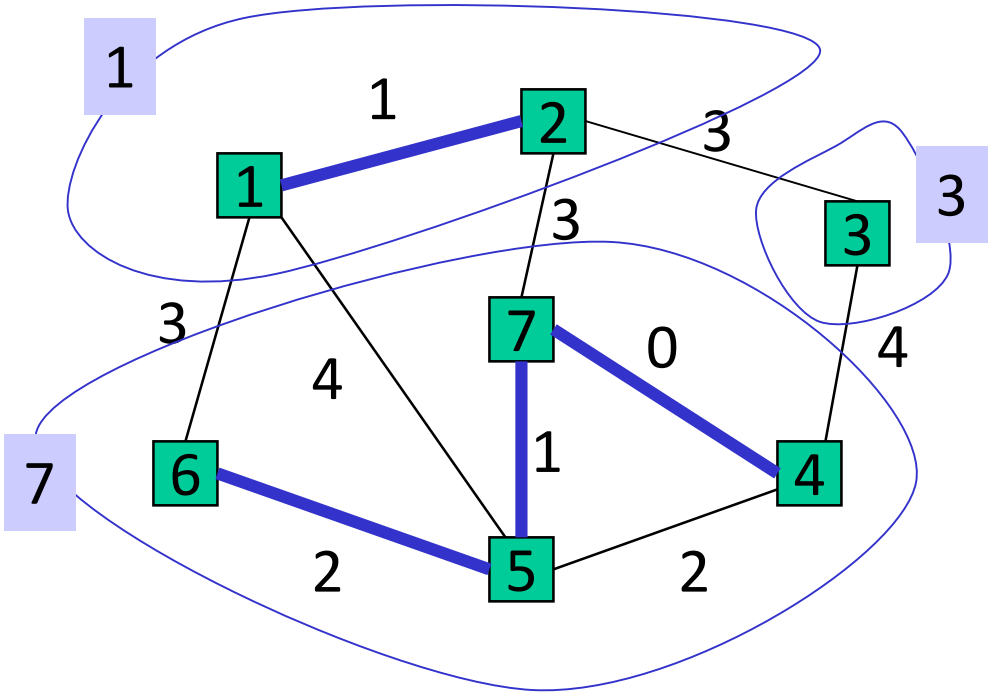


S

{1,2,3,4,5,6,7,… 36}

E

Back to Kruskal: Example of DU/F (1)



Find(5) = 7
Find(4) = 7

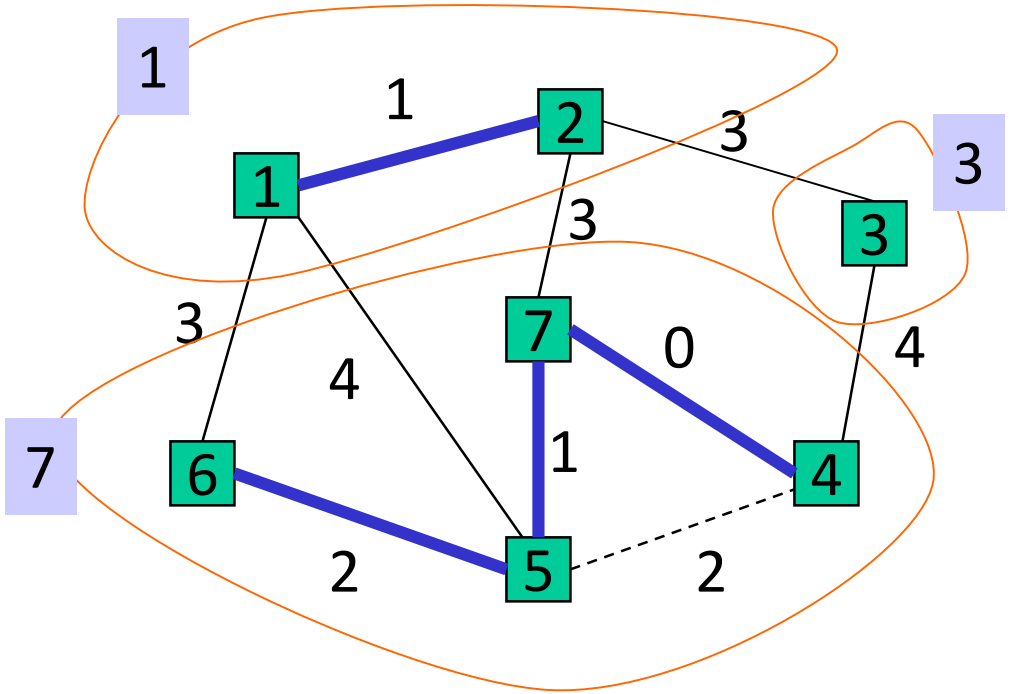
{7,4} {2,1} {7,5} {5,6} {5,4} {1,6} {2,7} {2,3} {3,4} {1,5}
0 1 1 2 2 3 3 3 4 4

u,v in the same set → (u,v) is not added to T

Back to Kruskal

- The sets correspond to components of the graph that are connected by tree edges.
- Initially, there are no edges and each vertex form a connected component.
- Each time an edge e=(v,u) is considered, the algorithm checks the components of u and v.
- If they are in the same set, then adding e will close a cycle. Otherwise, e can be added to the spanning tree.

Example of DU/F (2)



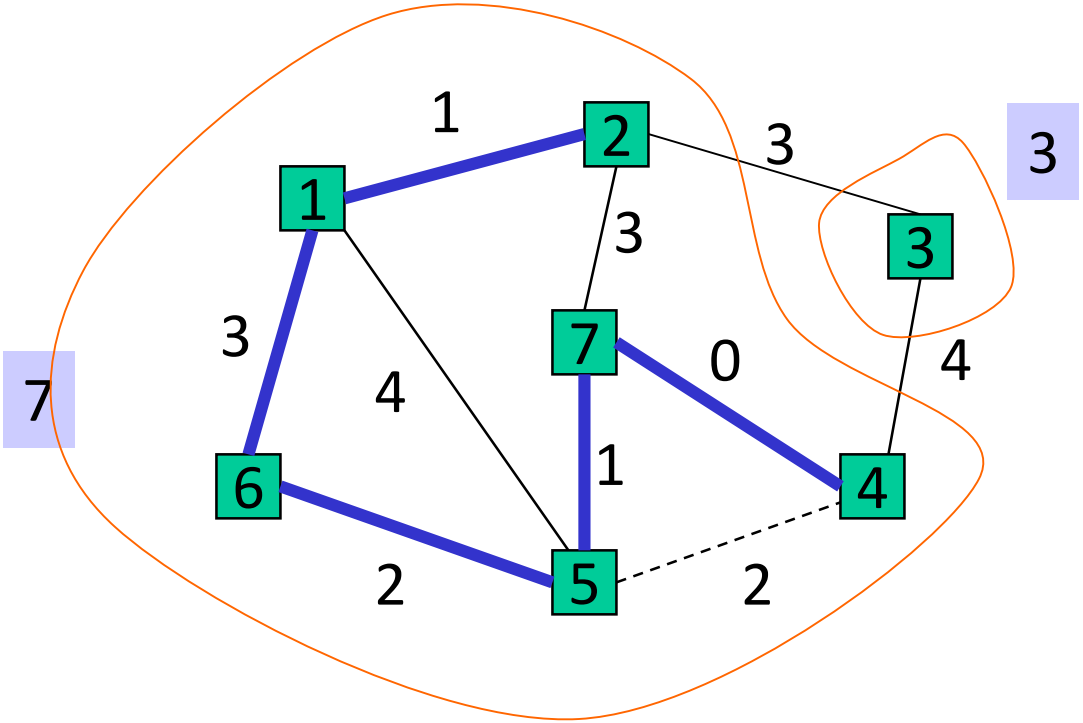
Find(1) = 1
Find(6) = 7

{7,4}	{2,1}	{7,5}	{5,6}	{5,4}	{1,6}	{2,7}	{2,3}	{3,4}	{1,5}
0	1	1	2	2	3	3	3	4	4

u,v in different sets → add (u,v) to T, union the sets.

Example of DU/F (3)

Union(1,7)



{7,4} {2,1} {7,5} {5,6} {5,4} {1,6} {2,7} {2,3} {3,4} {1,5}

0 1 1 2 2 3 3 3 4 4

Kruskal's Algorithm with DU / F

```
Sort the edges by increasing cost;
Initialize T to be empty;
for each edge {i,j} chosen in increasing order do
    u := Find(i);
    v := Find(j);
    if (u ≠ v) then
        add {i,j} to T;
        Union(u,v);
```

Amortized Complexity

- Disjoint union/find can be implemented such that the average time per operation is essentially a constant.
- An individual operation can be costly, but over time the average cost per operation is not.
- On average, each U/F operation takes $O(\alpha(m,n))$ time.

Inverse Ackermann function. Practically, this is a constant.

Evaluation of Kruskal

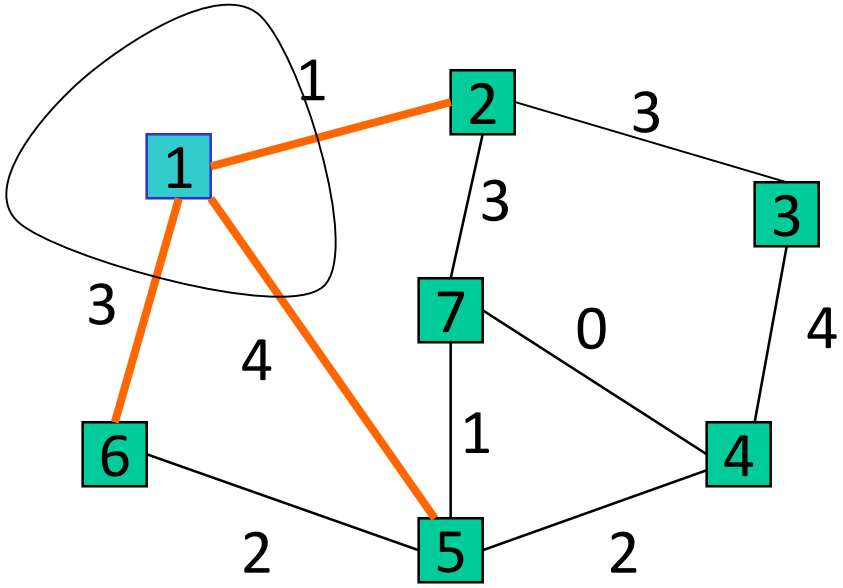
- G has n vertices and m edges.
- Sort the edges - O(m log m).
- Traverse the sorted edge list using efficient UF - O(m α(m,n)).
- Total time is O(m log m).

Prim's Algorithm

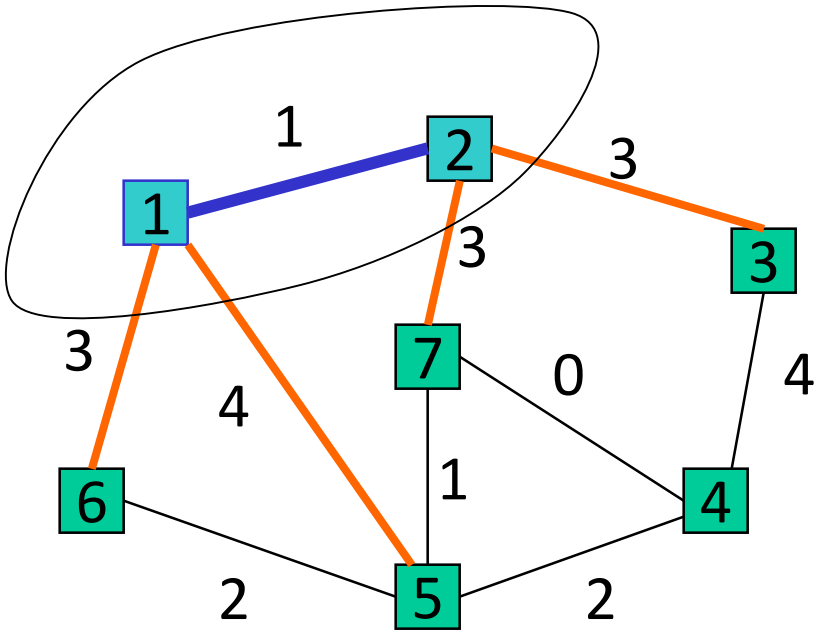
- Another algorithm that follows the blue/red rules.
- During the algorithm, the blue edges form a single tree T.
- Every vertex is either 'already in the blue tree' or 'isolated' (none of its adjacent edges is blue).
- Note: In Kruskal's algorithm we may have a forest of many blue trees.
- In Prim's algorithm, initially, the blue tree consists of one vertex and no edges.
- In every iteration we apply the blue rule. The cut is defined by the vertices of the blue tree.

Prim's Algorithm

Example: Assume we begin the run with node 1

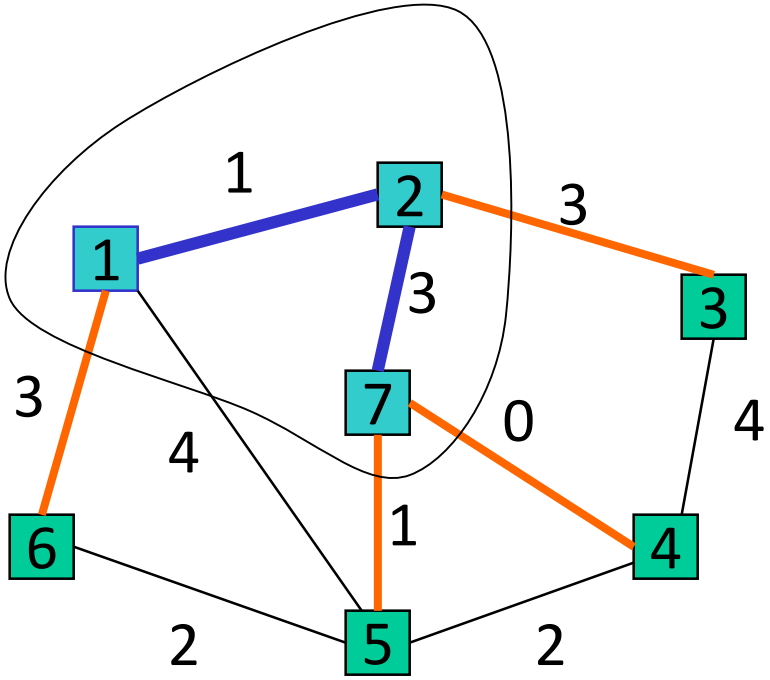


Prim's Algorithm 2



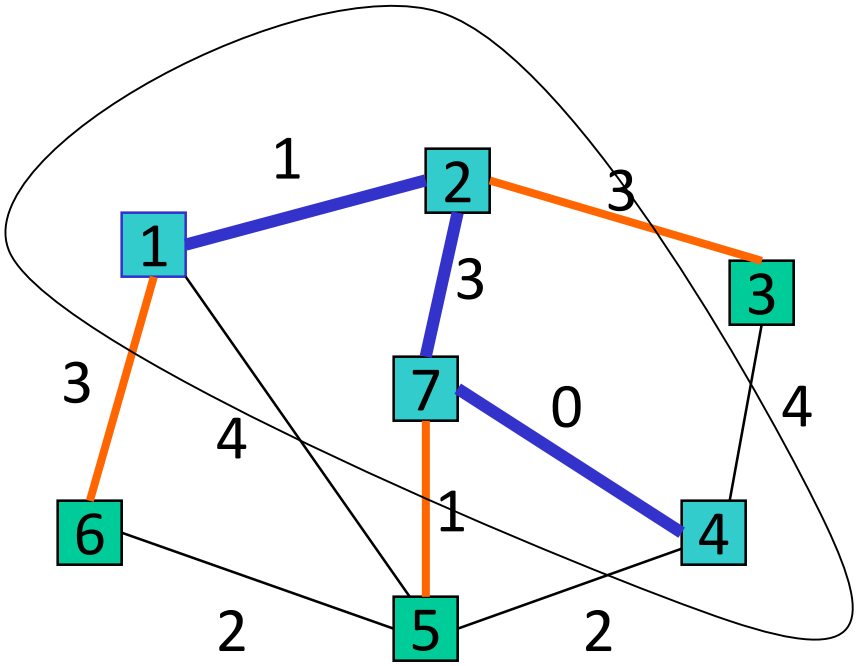
Add to T the cheapest edge connecting T with the rest of the graph.

Prim's Algorithm 3

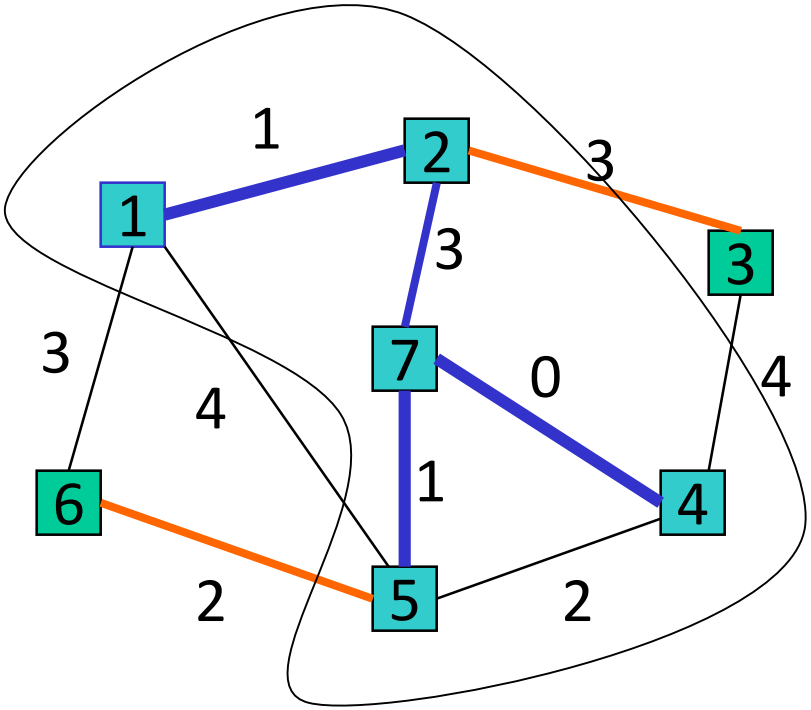


For each vertex not in the tree maintain the cheapest edge to a vertex in the tree (if exists).

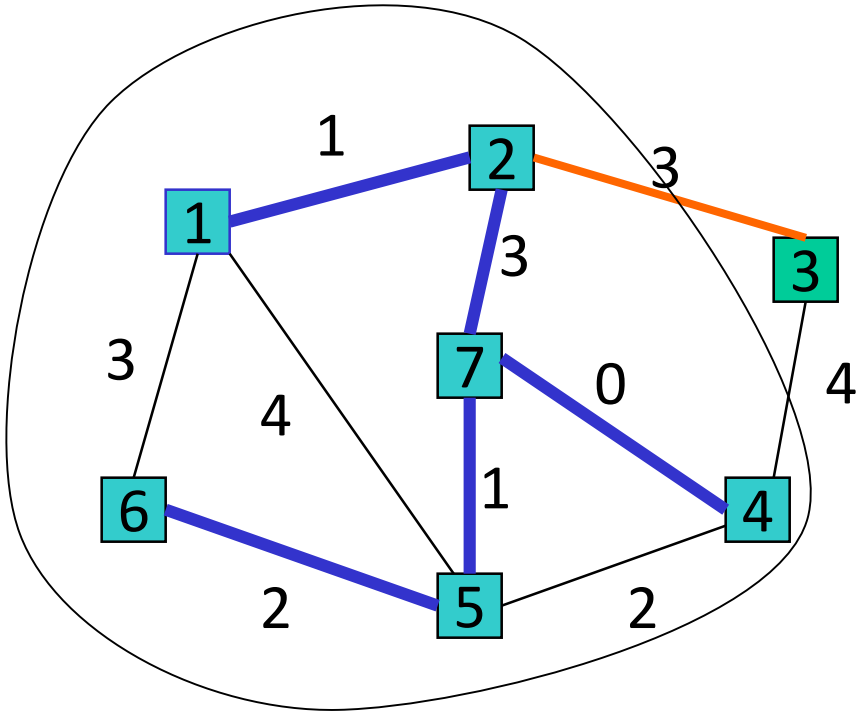
Prim's Algorithm 4



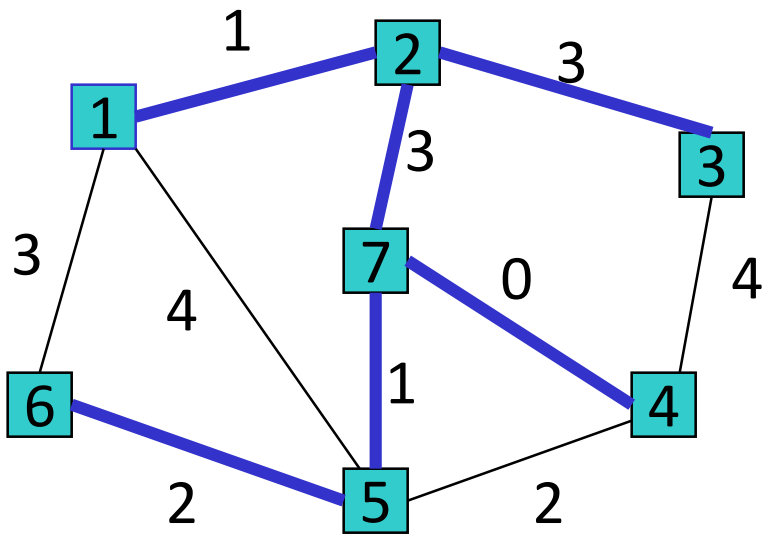
Prim's Algorithm 5



Prim's Algorithm 6



Prim's Algorithm 7

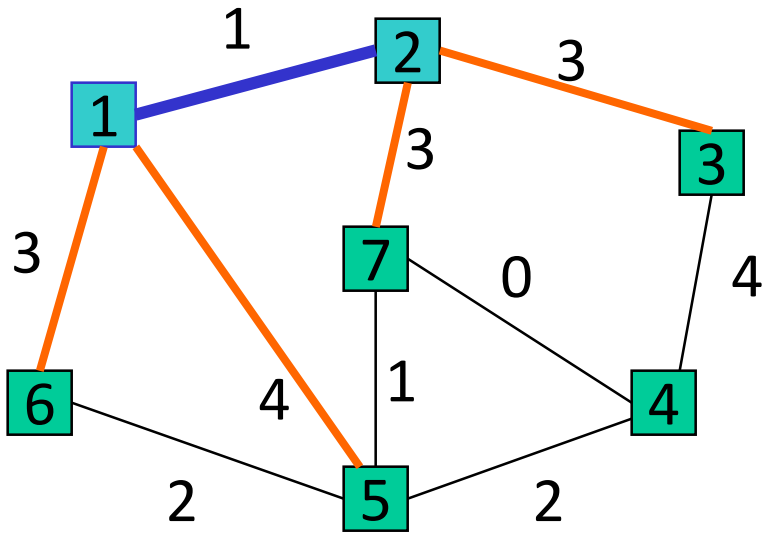


Correctness Proof for Prim

- Repeatedly executes the blue rule (n-1 times).
- In each step we consider the cut defined by the vertices that are already in T.

Data Structures for Prim

1. The graph is given by Adjacency Lists – when a node joins T, we need to look at all the edges adjacent toit
2. Array for the best edges to the tree.

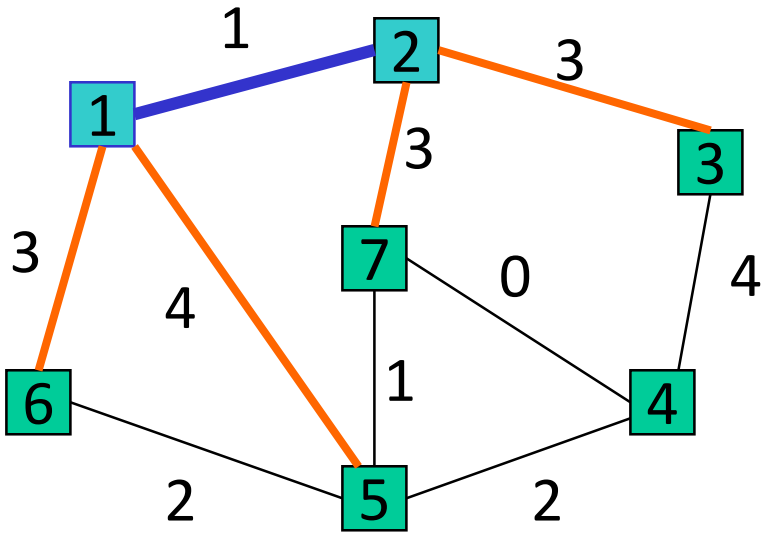


	1	2	3	4	5	6	7
to			2	-	1	1	2
cost			3	∞	4	3	3

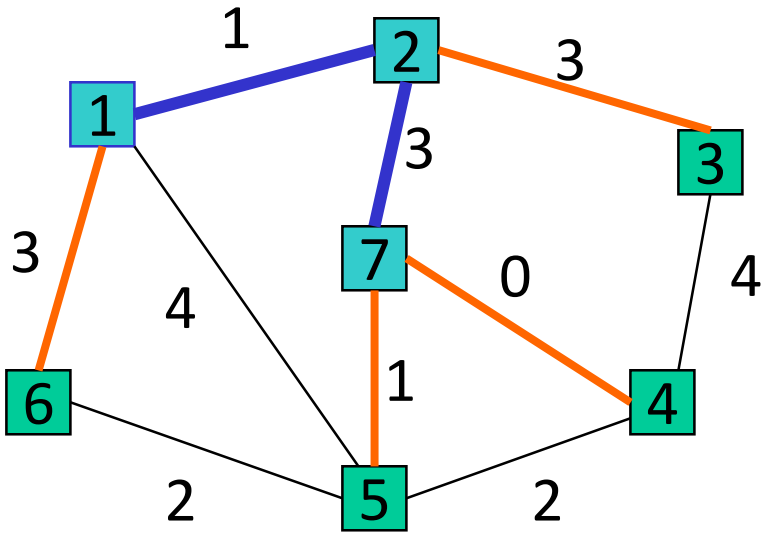
(1,5) is the cheapest edge in the cut adjacent to 5. Its cost is 4.

Data Structures for Prim

3. Priority queue for all edges in the best-edge array (orange edges, from isolated nodes to the tree).
   – extract-min, decrease-key (e.g. binary heap).

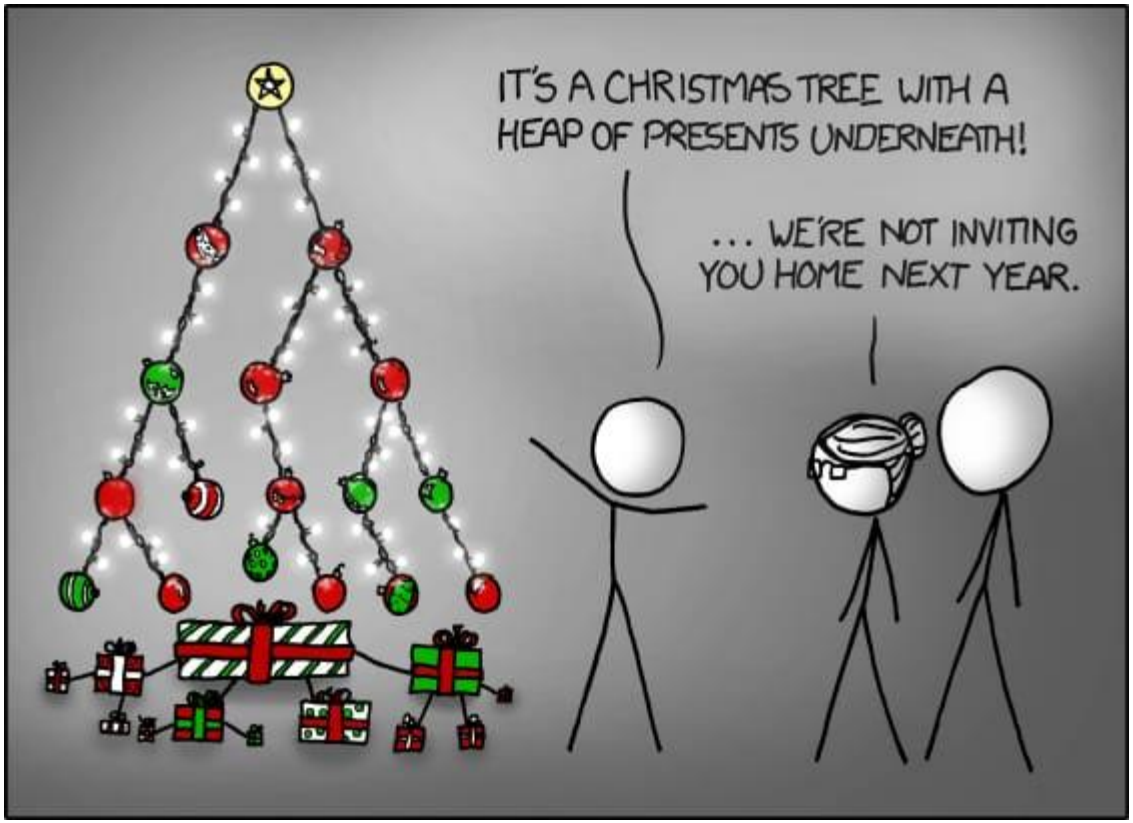


	1	2	3	4	5	6	7
to			2	-	1	1	2
cost			3	∞	4	3	3



	1	2	3	4	5	6	7
to			2	7	7	1	
cost			3	0	1	3	

Evaluation of Prim



Evaluation of Prim

- Priority queue: O(log n) per operation, using a binary heap.
- O(m+n) priority queue operations:
  - An edge is considered when a vertex incident to it joins the tree.
  - Every node is added to the tree once.
- Time complexity is O((m+n) log n)
  [= O(m log n) we assume G is connected]
- Can be reduced to O(m + n log n) when using Fibonacci heaps

Prim's Algorithm

1. for each u ∈ V:
2. u.key ← ∞
3. u.parent ← null
4. r.key ← 0
5. Q ← V
6. while Q is not empty:
7. u ← extract-min(Q)
8. for each v adjacent to u:
9. if v in Q and c(u,v) < v.key:
10. v.parent ← u
11. v.key ← c(u,v)
12. T = {(v, v.parent) | v ∈ V\{r} }

Initialization
r is the (arbitrary selected) first node.

Q is a min priority queue

iterations

Kruskal vs Prim

demo1
demo2

- Kruskal
  - Simple
  - Good with sparse graphs - O(m log m)
- Prim
  - More complicated
  - Perhaps better with dense graphs - O(m log n)

Note: O(log n) = O(log m)

($n-1 \leq m < n^2$ implies $\log m < \log n^2 = 2 \log n$)

Sample Problem 1

Let G be a connected undirected graph with positive edge weights. An edge is considered *cheap* if its weight is less than 10.
It is known that for two vertices $a,b \in V$, there exists a path connecting a and b consisting of cheap edges.

Prove or give a counter example: In every MST of G, the (only) path connecting a and b consists of cheap edges.

Sample Problem 1

Proof: Let T be an MST of G. Consider the run of Kruskal that produces T (in the recitation, you will prove that such a run exists for any MST T).
Consider the time-point after all cheap edges were considered. We show that a and b are already connected at this time-point – implying that the path connecting them is as required.

$$c(e_1) \leq c(e_2) \leq \cdots \leq c(e_m)$$



cheap edges –
already considered

Non-cheap edges –
not considered yet

Sample Problem 1

Assume by contradiction that a and b are not in the same connected component. Let p be a path connecting a and b consisting of cheap edges. Since a and b are not in the same component, there must be an edge in p whose endpoints are in different components. This edge was considered already (weight < 10) and was supposed to be included in the MST – a contradiction to Kruskal's operation.



Sample problem 2

Given is an undirected connected graph $G=(V,E)$, with positive edge weights. Let C be the maximal cost of an edge, that is, $\forall e \in E,\ w(e) \leq C$ and for at least one edge $w(e)=C$.

Is it true that every MST of G includes the same number of edges of weight C?

If **YES**, describe an algorithm, as efficient as you can, that returns the number of edges of weight C in every MST of the graph.

If **NO**, give a counter example.

Sample problem 2

Yes.

Algorithm:

1. Remove from the graph edges having weight C.
2. Use DFS or BFS to count the number, x, of connected components in the resulting graph.
3. Return x-1

Details of step 2: Loop through the vertices, starting a new BFS or DFS whenever the loop reaches a vertex that has not been included in a previously found connected component (not marked by BFS or DFS). Increase x by 1 with every new BFS/DFS (new component).

Sample problem 2

Correctness: Assume that Kruskal's algorithm is performed on the graph. Consider the time after all edges having weight less than C were considered.
The number of Union/Find sets at this time point equals x, because any two vertices that are connected by edges having weight less than C are in the same component (by the exercise last week).

Note also that these components are the same for any run of Kruskal, and therefore for any MST. In the MST itself (after Kruskal terminates), there is a single component, therefore, x-1 edges of weight C should be added in order to merge the x components into a single one.

Sample problem 2

Complexity:

Assume the graph is given by adjacency lists:

Step 1: Scan the adj. lists to produce the graph of cheap edges: O(|E|)

Step 2: DFS or BFS + marking : O(|E|)

Step 3: O(1).

All together: O(|E|).